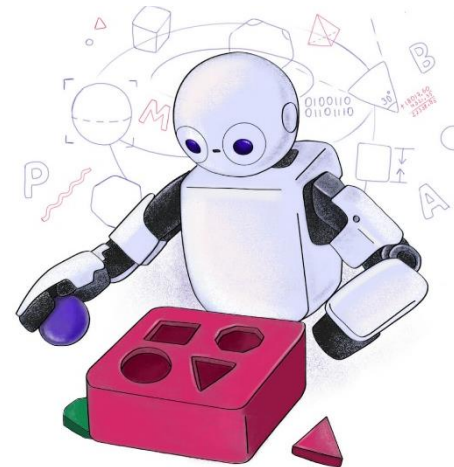


TP557 - Tópicos avançados em IoT e
Machine Learning:

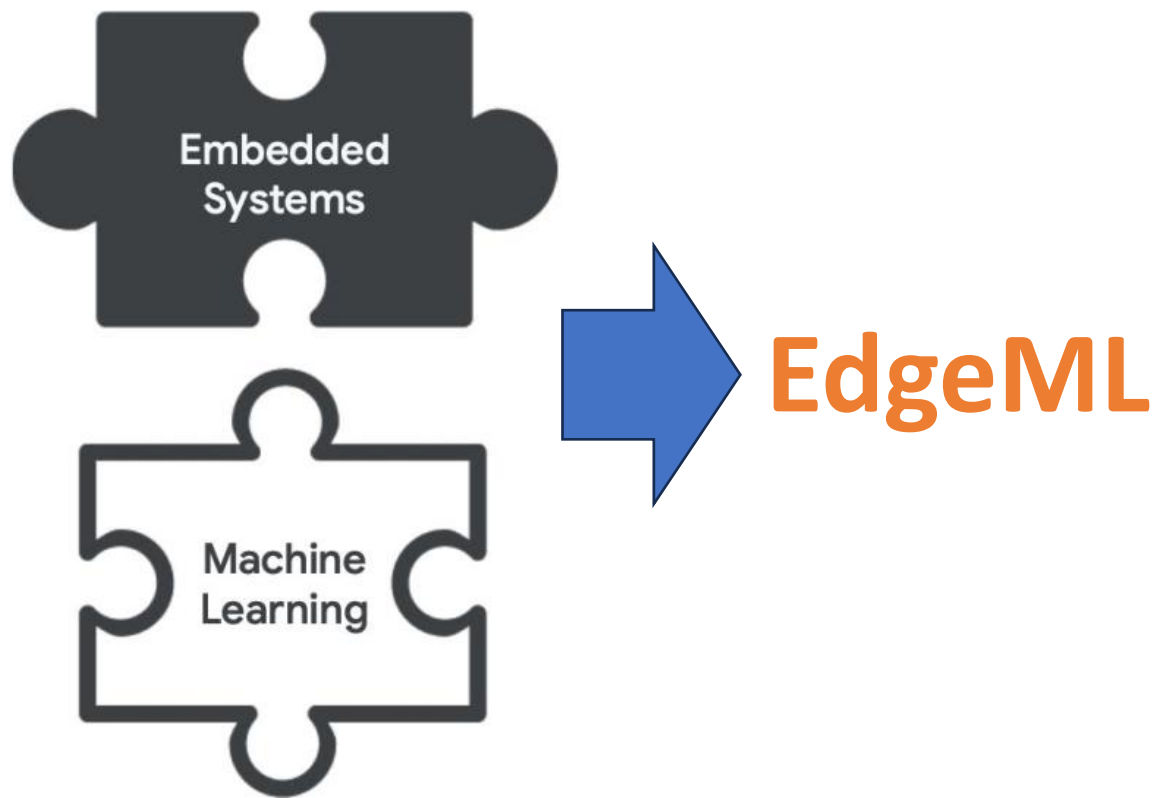
*Desafios do TinyML:
Sistemas Embarcados*



Inatel

Felipe Augusto Pereira de Figueiredo
felipe.figueiredo@inatel.br

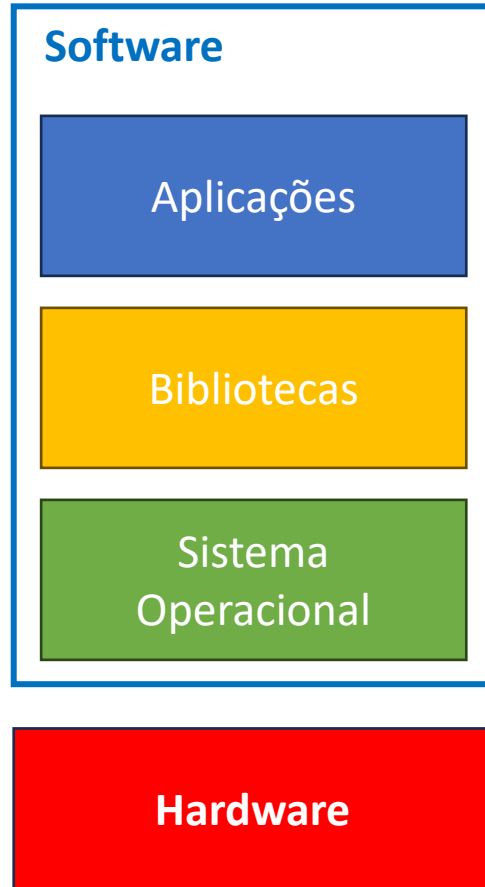
Recapitulando



- Vimos anteriormente que *Edge(Tiny)ML* é a **junção de ML com sistemas embarcados**.
- Ou seja, é a execução de algoritmos de ML em sistemas com **restrições de processamento, memória e consumo de energia**.
- Nesta aula, falaremos sobre os **desafios de hardware (HW) para execução de aplicações de ML nesses dispositivos** com restrições.

Quais desafios encontraremos na
execução de *machine learning* em
sistemas embarcados?

Sistema computacional de uso geral



- Antes de falarmos de sistemas *embarcados*, vamos fazer um *paralelo* com o *sistema de um computador de uso geral*.
- Em termos gerais, o que temos é o software (SW) (aplicações, bibliotecas e sistema operacional (SO)) rodando em cima do HW.

Hardware

CPU



Armazenamento
de curto prazo



Armazenamento de longo prazo



- Os componentes mais importantes do HW de um computador de uso geral são:
 - Unidade de processamento central (CPU), o *processador*.
 - E as memórias de armazenamento volátil e de longo prazo.

Hardware

CPU



Armazenamento
de curto prazo



Armazenamento de longo prazo



- **Unidade de processamento central (CPU):** é o ***cérebro do computador.***
- Ela é ***responsável pela execução de programas:***
 - cálculos,
 - tomada de decisões,
 - controle de dispositivos.

Hardware

CPU



Armazenamento
de curto prazo



Armazenamento de longo prazo



- **Memória volátil:** *armazena temporariamente dados e instruções sendo processados pela CPU.*
 - Exemplos: RAM e a cache do processador.

Hardware

CPU



Armazenamento
de curto prazo

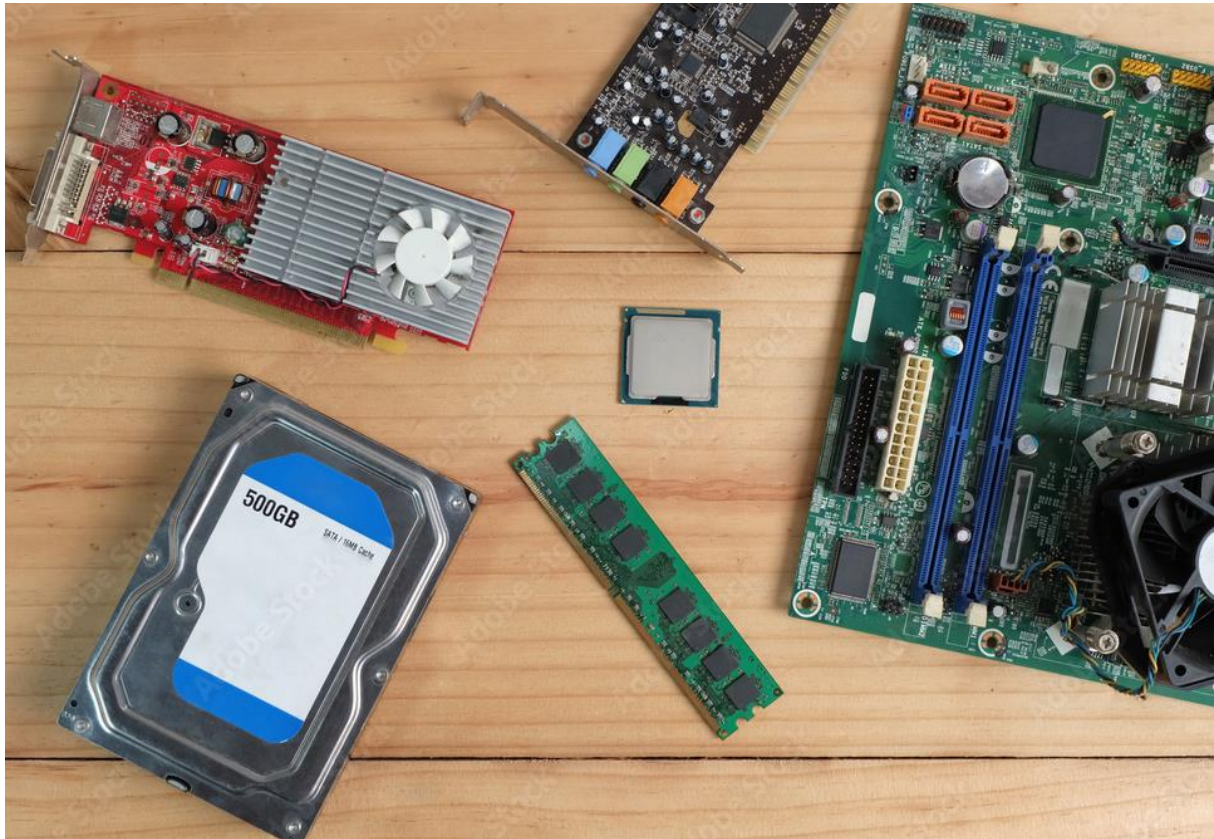


Armazenamento de longo prazo



- **Memória não-volátil:** *armazena permanentemente arquivos*, programas e outros dados que podem ser acessados posteriormente mesmo após sistema ser reiniciado.
 - Exemplos: discos rígidos (HDs), unidades de estado sólido (SSDs), memória *flash* e memória de acesso aleatório não volátil (NVRAM).

Hardware

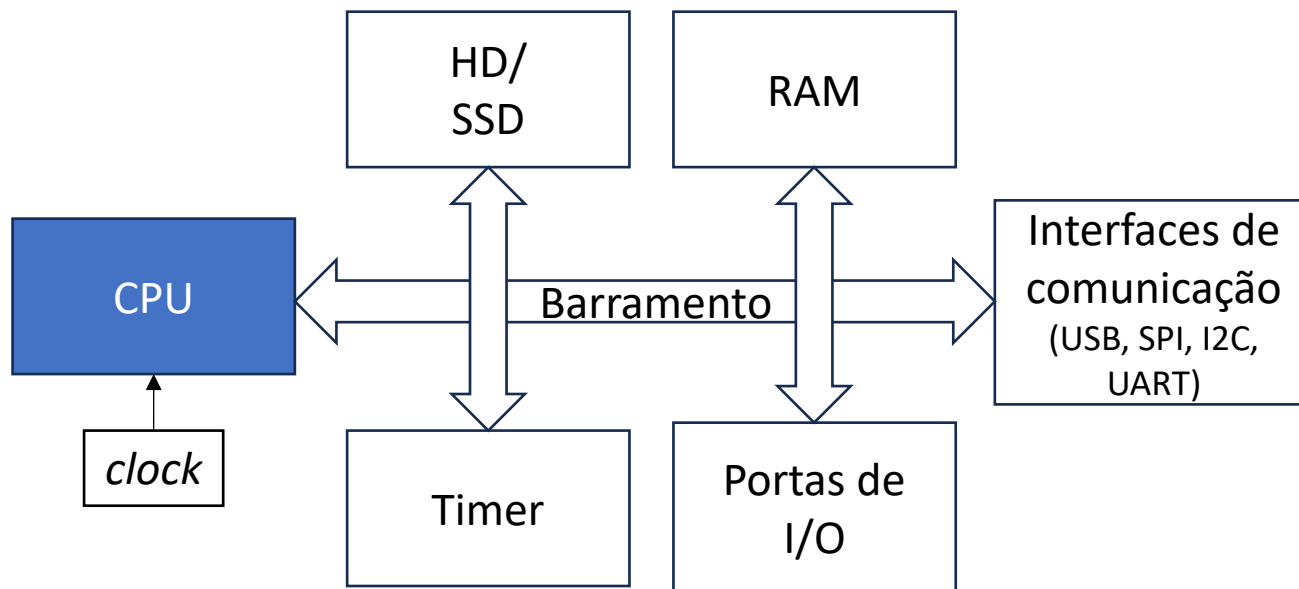


- Mas percebam que todos eles são ***componentes separados.***
- Discutiremos mais sobre isso na sequência

***Porém, em geral, sistemas
embarcados possuem
microcontroladores!***

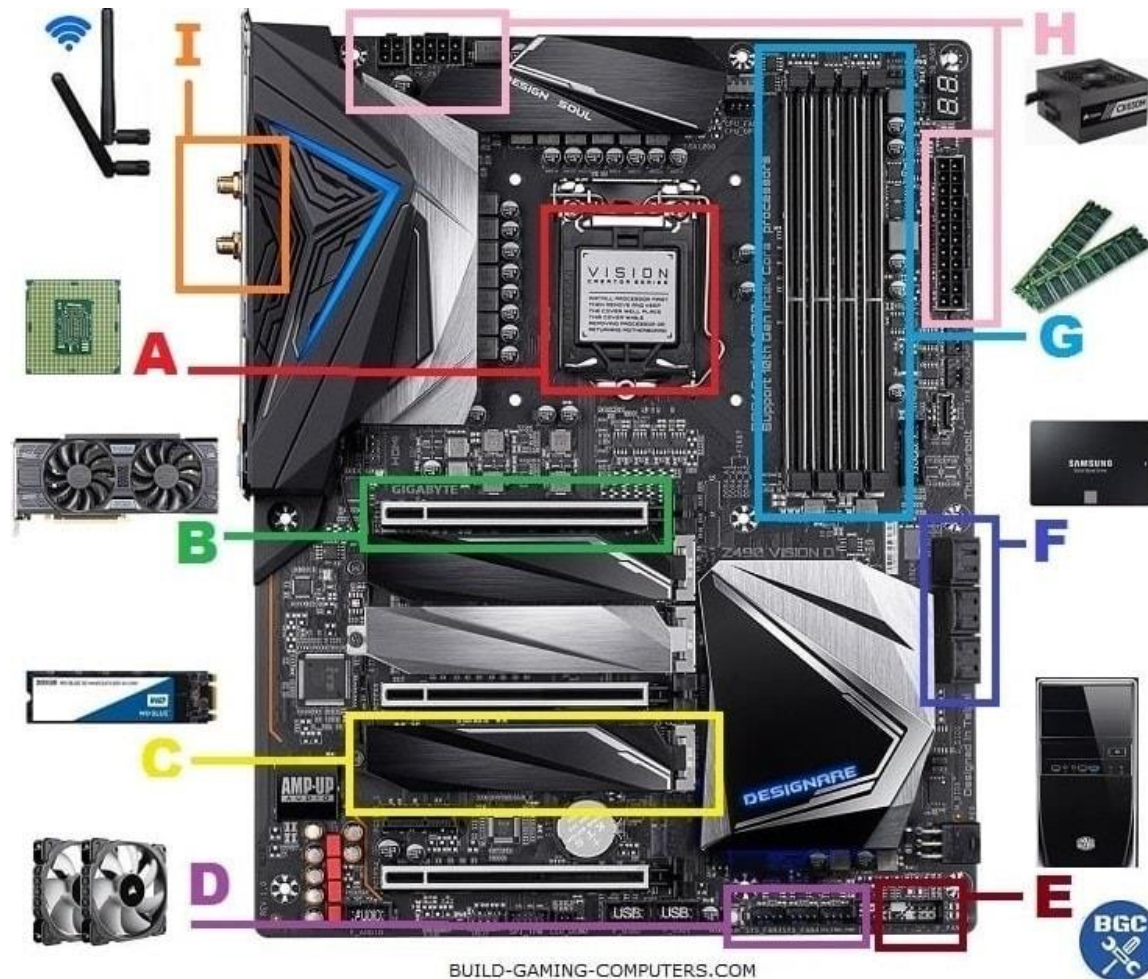
Micro*processador*
versus
Micro*controlador*

Microprocessador: apenas uma peça do quebra-cabeças



- A CPU é a ***parte principal do computador***, mas para que ela funcione, ***são necessários outros componentes***, como:
 - sistema de *clock*
 - memórias volátil e não-volátil,
 - controladores de I/O,
 - interfaces de comunicação,
 - timers,
 - etc.

Microprocessador: apenas uma peça do quebra-cabeças

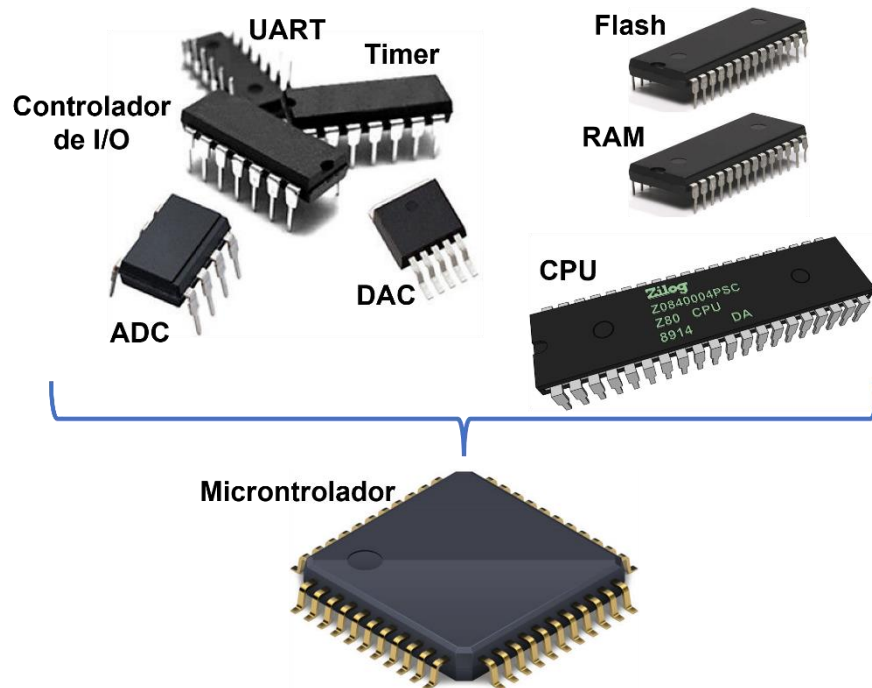


- Em um computador de uso geral, *todos estes componentes são chips separados* soldados ou conectados à placa mãe e *interligados através do barramento* de comunicação disponibilizado pela *placa mãe*.

Microcontrolador, tudo junto e misturado

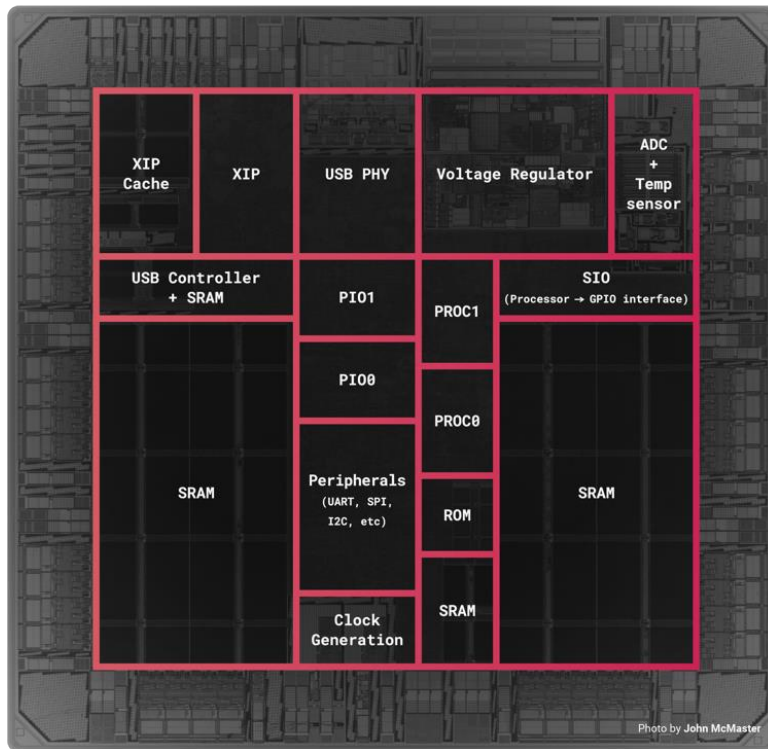
Microcontrolador

CPU	HD/ SSD	RAM
Timer	Portas de I/O	Interfaces de comunicação



- O microcontrolador é um ***dispositivo completo em um único chip***.
- Ele combina uma CPU com memórias, periféricos de entrada/saída, temporizadores e outros recursos, tornando-o ***adequado para aplicações embarcadas***.
- O ***barramento é interno ao chip***.

Microcontrolador, tudo junto e misturado



- **Die** (ou pastilha) do microcontrolador RP2040.
 - *Die* é bloco de material semicondutor onde um circuito é fabricado.
- Primeiro **microcontrolador** projetado pela **Raspberry Pi Ltd**.
- Contém um ARM Cortex-M0+ dual core de 32 bits.
- Percebam que ele possui os componentes necessários para funcionar integrados em um pequeno *chip*.



Microprocessador

- **Cérebro** de um *sistema de uso geral*.
- É **apenas a CPU**, memória, armazenamento, etc. são externos a ele.
- Usado principalmente *sistemas de uso geral* como desktops, laptops e servidores.
- Oferece **flexibilidade** porque é projetado para ser **usado em uma ampla variedade de aplicações**.
- O **sistema é grande** e, em geral, **não apresenta restrições** de processamento, memória, armazenamento e consumo de energia.

Microcontrolador

- **Cérebro** de um *sistema embarcado*.
- **Memória, armazenamento e outros periféricos** (e.g., controladores de I/O, ADC, DAC, timers, etc.) **são internos ao chip**.
- Usado principalmente em *sistemas especializados*, com função específica, como MP3 players, telefones, sensores, etc.
- **Não oferece flexibilidade** porque é projetado para atender a um **propósito específico**.
- O **sistema é pequeno** e **apresenta restrições** de processamento, memória, armazenamento e consumo de energia.

Diferenças de magnitude

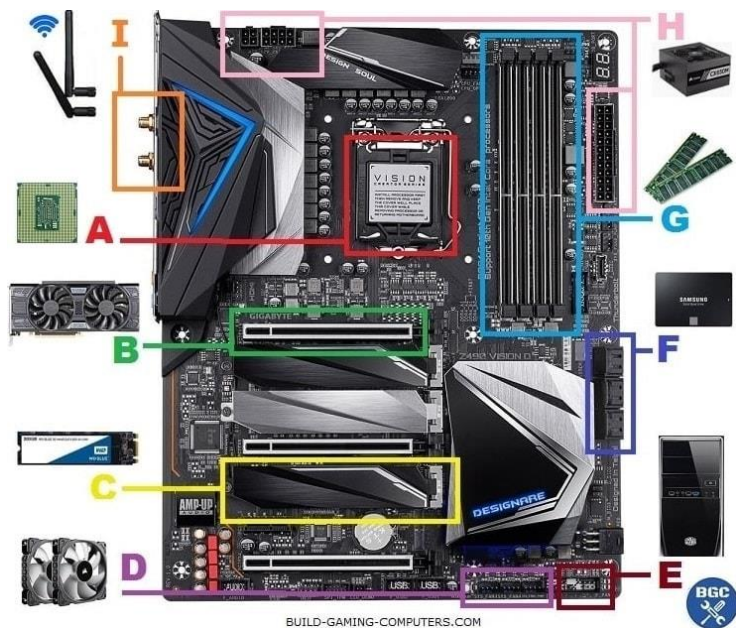
	Microprocessador	>	Microcontrolador
Freq. de operação ➡	1 GHz – 4 GHz	> 2.5 x	1 MHz – 400 MHz
Memória RAM ➡	512 MB – 128 GB	> 1000 x	2 KB – 512 KB
Armazenamento ➡	64 GB – 16 TB	> 30000 x	32 KB – 2 MB
Consumo ➡	30 – 700 W	> 1000 x	150 μ W – 24 mW



nRF52840*
64 MHz
256 KB
1 MB
< 1 mW

* μ C usado
no kit tinyML

Diferenças de magnitude



- Quanto *mais elevada é a frequência de operação* de uma CPU, *mais instruções por segundo são processadas* e, consequentemente, *maior é o consumo de energia*.
- As diferenças são muito grandes, principalmente devido ao tamanho dos sistemas e suas limitações.
- Percebiam que *vários desafios devem ser superados* para que aplicações de ML possam ser executadas nesses dispositivos com várias restrições.

Portanto, ***vários desafios devem ser superados***
para que aplicações de ML possam ser
executadas nesses dispositivos com restrições
computacionais.

Quais são as implicações das limitações?



nRF52840
64 MHz
256 KB
1 MB
< 1 mW

- Com **sistemas computacionais de uso geral**, nós praticamente **não precisamos nos preocupar com memória, consumo, poder computacional**.
- Porém, com sistemas embarcados precisamos saber
 - o **quanto de memória a aplicação requer**, pois pode não caber na memória.
 - o **consumo de energia da aplicação**, pois a bateria pode não durar muito.
 - a **quantidade necessária de processamento de dados requerido pela aplicação**, pois a CPU pode não atender.

Comparação de hardwares embarcados

Os **endpoints** são, em geral, dispositivos com restrições de recursos.



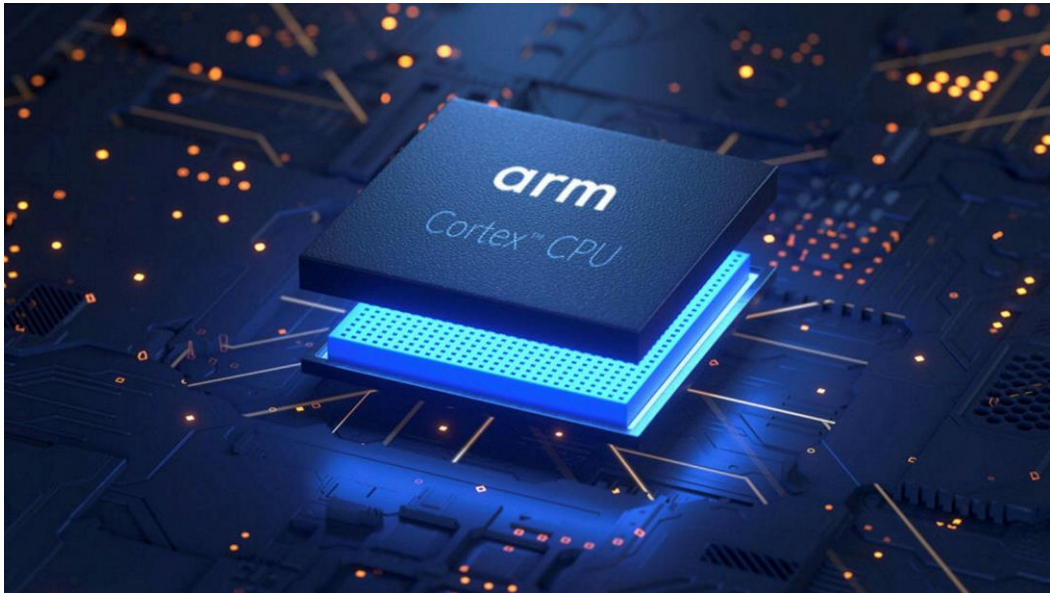
	Raspberry Pi Pico	Arduino Nano Sense	ESP 32	Seed Wio Terminal	Arduino Portenta
CPU*	Dual Core ARM Cortex M0+	Single Core ARM Cortex M4	Dual Core Cadence Xtensa LX6	Single Core ARM Cortex M4	Dual Core ARM Cortex M4/M7
Clock	133 MHz	64 MHz	240 MHz	120 MHz	240/480 MHz
RAM	264 KB	256 KB	520 KB	192 KB	1 MB***
Flash	2 MB	1 MB	2 MB	4 MB	2 MB
Rádio	-	BLE**	BLE/WiFi	BLE/WiFi	BLE/WiFi
Sensores	Não	Sim	Não	Sim	Não
Preço	~ \$ 5,00	~ \$ 40,00	~ \$ 16,00	~ \$ 44,00	~ \$ 100,00

* Todas as CPUs são de 32 bits.

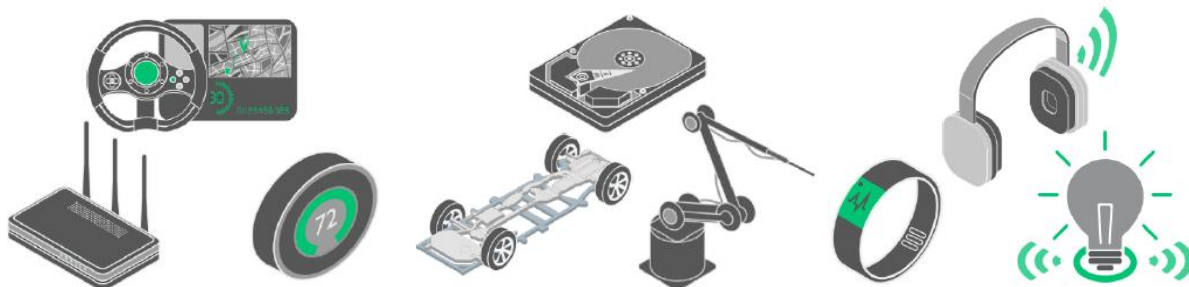
** Chip BLE multiprotocolo: suporta os protocolos ZigBee e Thread.

*** É possível rodar aplicações de detecção de objetos.

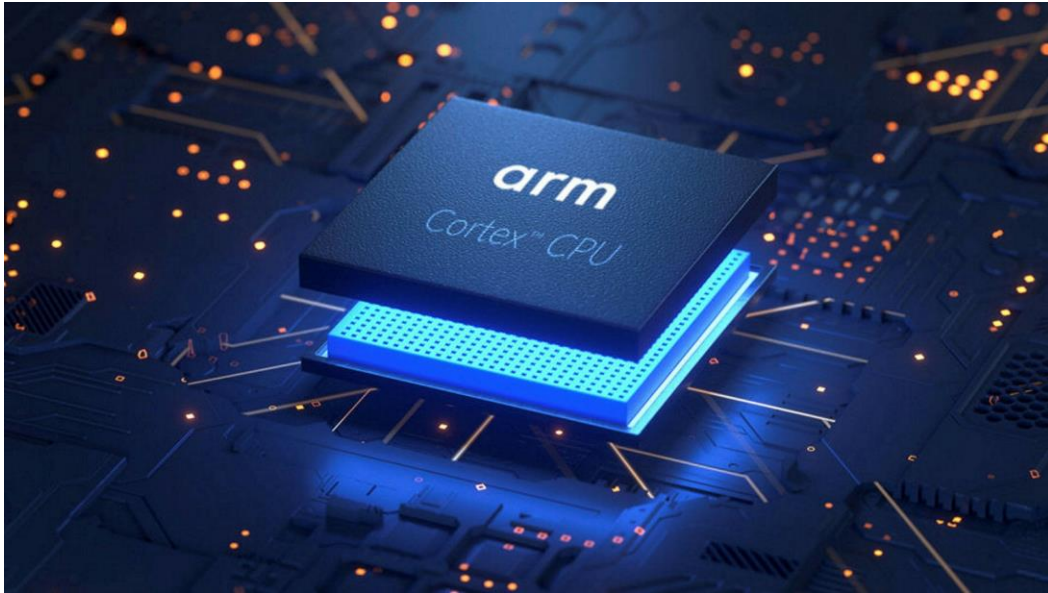
Dominância dos processadores ARM



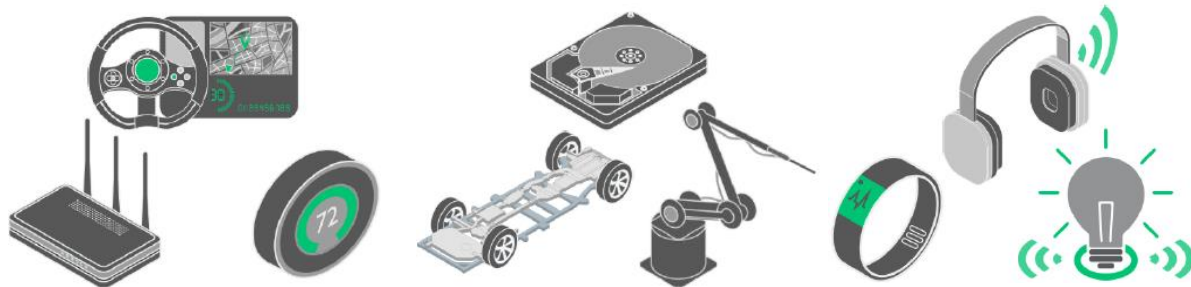
Processadores ARM estão em praticamente todos os sistemas embarcados.



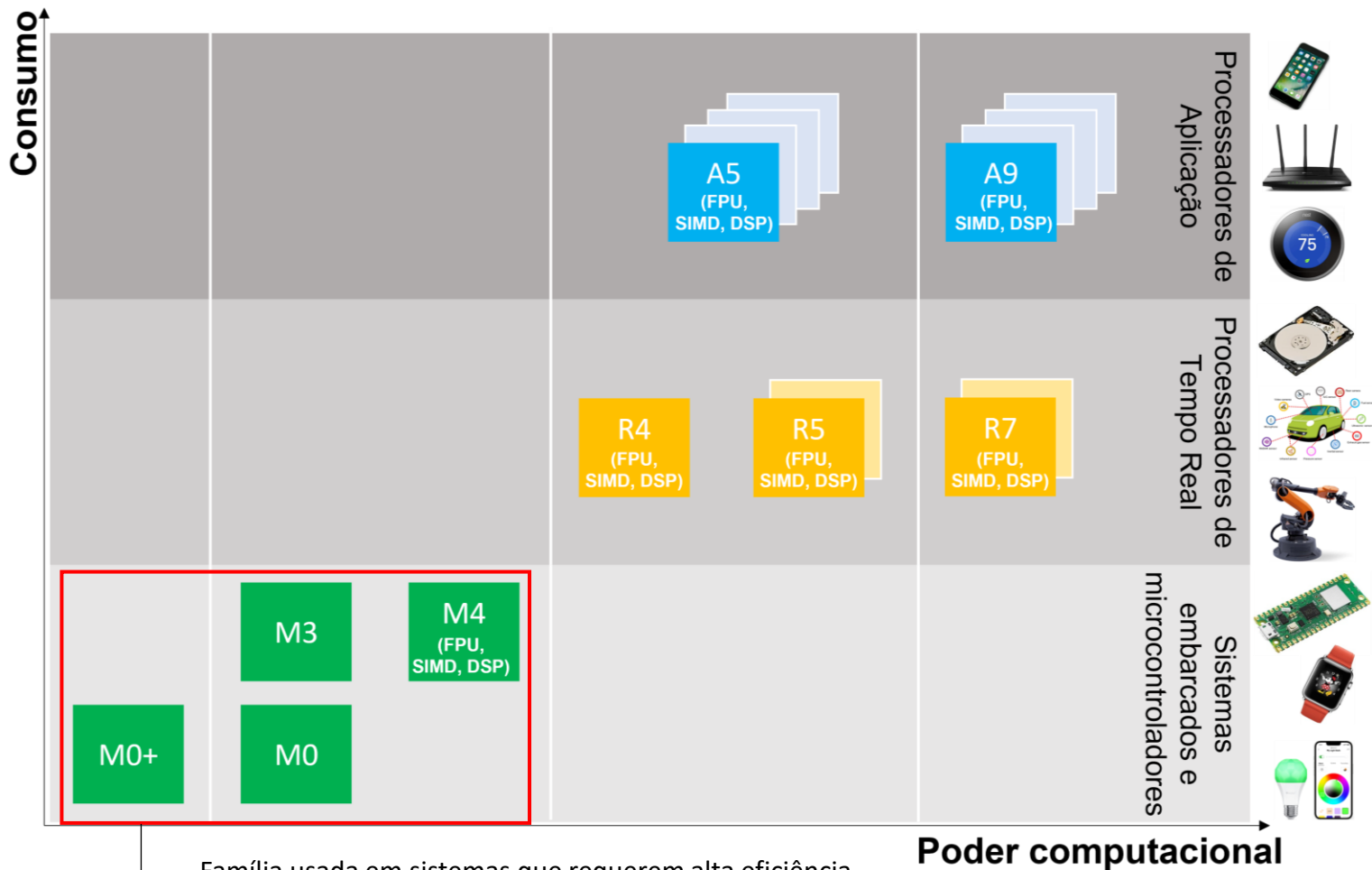
Dominância dos processadores ARM



- Isso se deve a alguns fatores principais:
 - **Alta eficiência energética:** consomem menos energia em comparação com outras arquiteturas de processadores.
 - **Baixo Custo:** como são focados em sistemas embarcados, têm alta demanda e com isso têm custo bem menor do que outros processadores.
 - **Flexibilidade:** fabricantes podem produzir *chips* personalizados para atender a necessidades específicas (número de núcleos, FPU, MMU, DSP, SIMD, etc.).



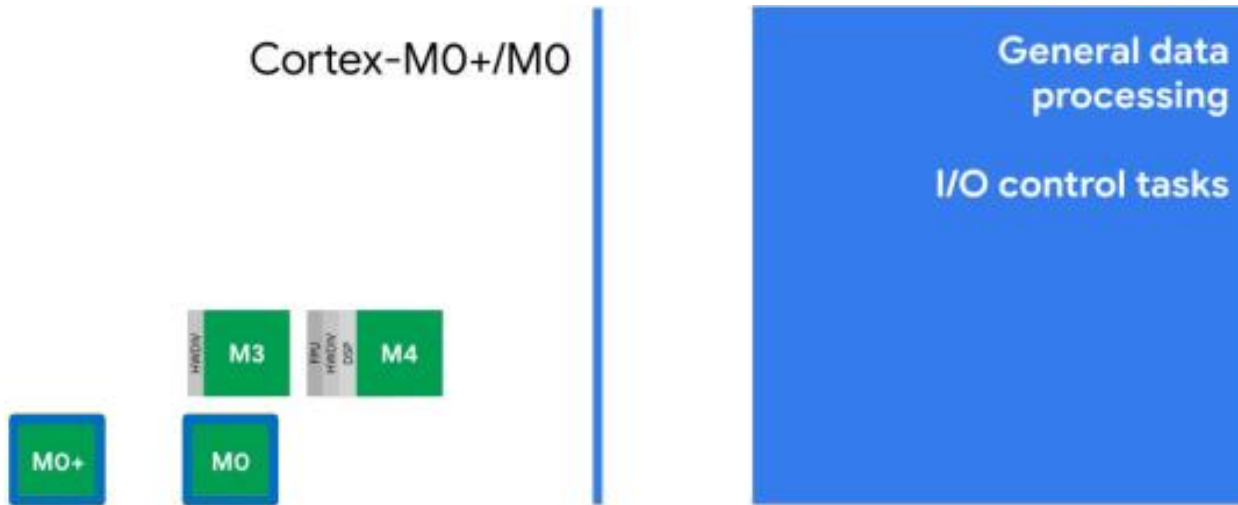
CPU ARM Cortex para sistemas embarcados



Família usada em sistemas que requerem alta eficiência energética. Porém, a alta eficiência energética implica em menor poder computacional.

- **Cortex A:** alta performance.
 - Suportam até 4 cores.
 - Por terem MMU, executam SOs completos, como Android e Linux.
- **Cortex R:** alta performance e resposta rápida.
 - Suportam até 2 cores.
 - Possuem recursos de redundância e recuperação de falhas.
 - Não têm MMU, assim, não executam SOs completos, apenas SOs mais simples como RTOSs.
- **Cortex M:** pequenos e com baixíssimo consumo.
 - Não executam SOs completos, mas podem executar RTOSs.

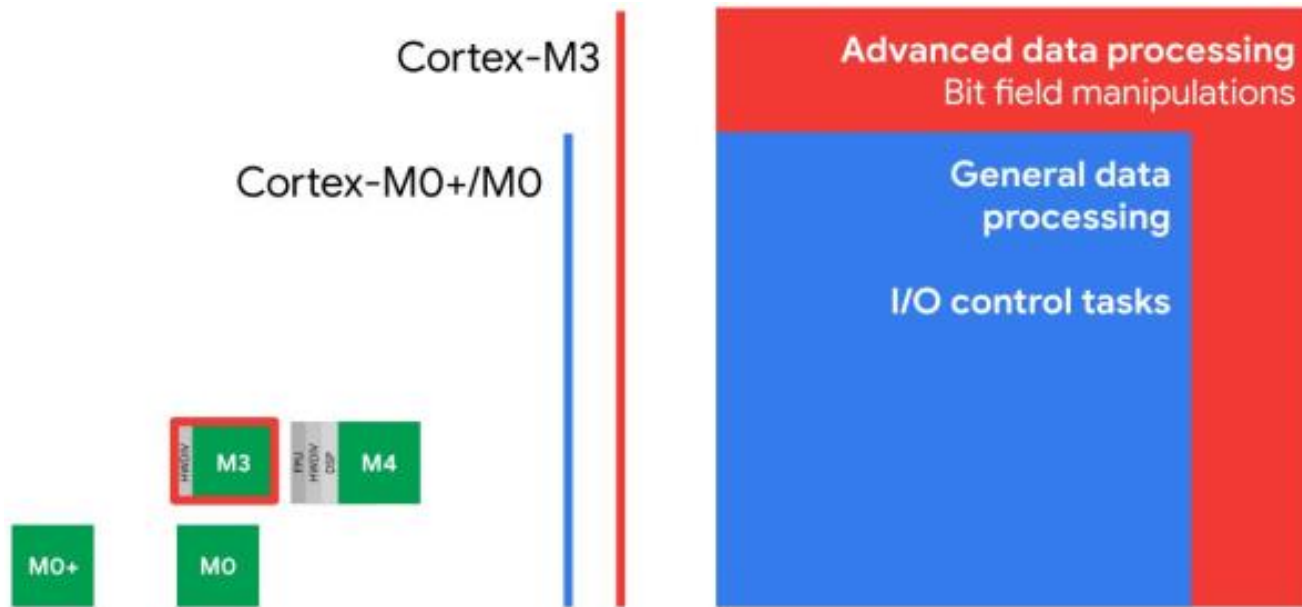
Conjunto de instruções da família ARM Cortex-M



Cortex-M0+ e M0

- Apresentam um **conjunto de instruções simplificado**, focando em **operações essenciais**, projetado para **economizar memória e energia em sistemas com recursos limitados**.
- Possui um conjunto de **instruções** para **processamento de dados de uso geral** e **tarefas simples de controle de E/S**.

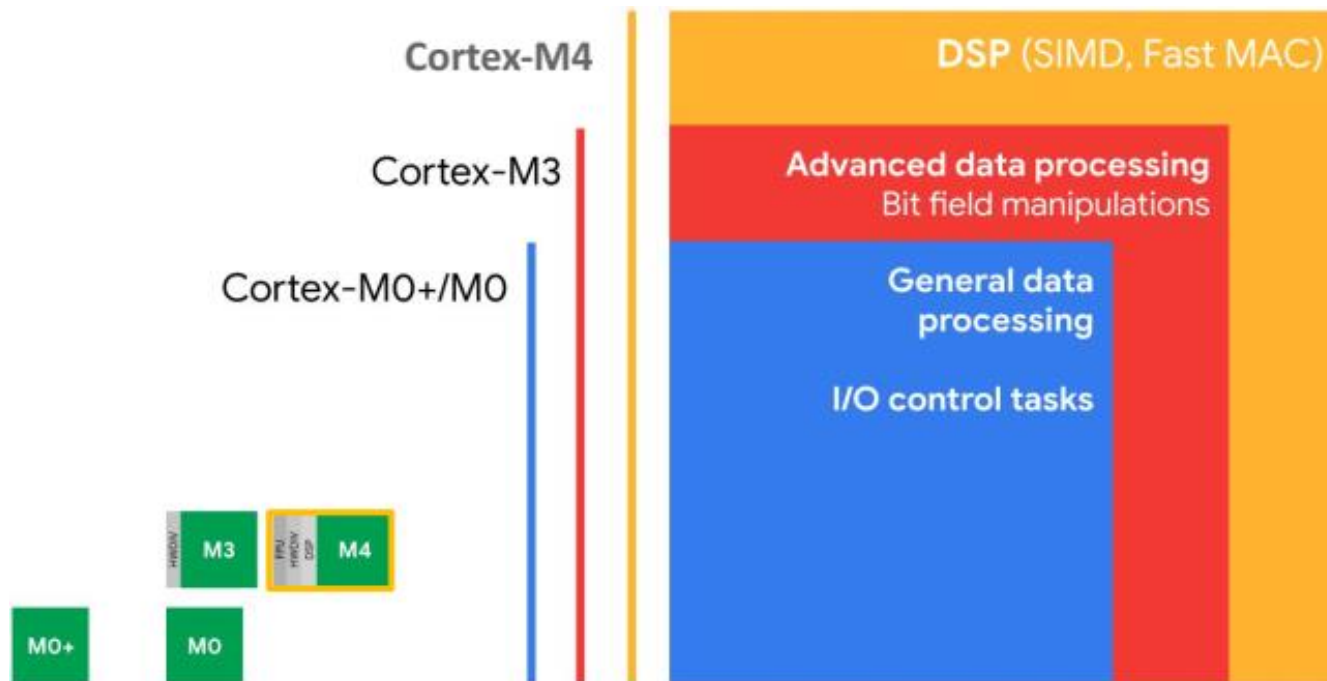
Conjunto de instruções da família ARM Cortex-M



Cortex-M3

- O conjunto de instruções aumenta com o modelo.
- Cada modelo mais alto oferece um ***superconjunto*** das instruções do modelo anterior.
 - Por exemplo, um M3 deve executar códigos do M0/M0+.
- Os Cortex-M3 ***adicionam*** instruções para ***manipulação de bits e divisão em hardware***.

Conjunto de instruções da família ARM Cortex-M



Cortex-M4

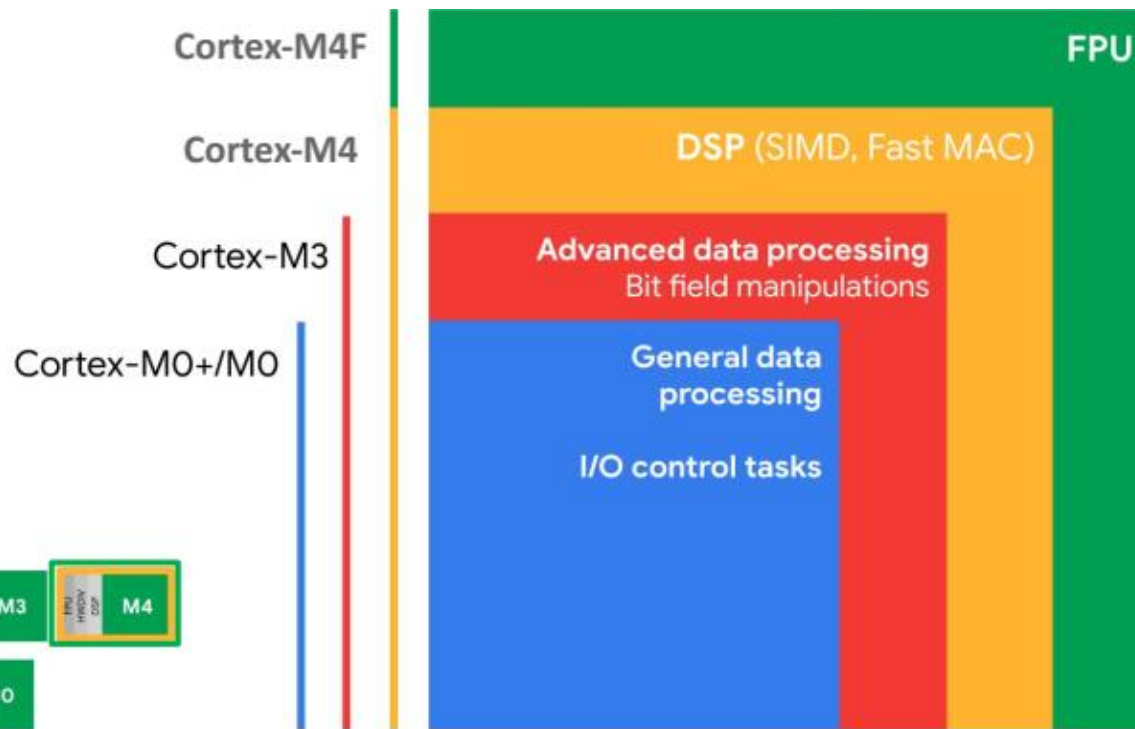
- Adicionam instruções básicas de processamento digital de sinais (DSP) como:
 - **Single Instruction Multiple Data (SIMD)**: a mesma instrução é aplicada a grandes quantidades de dados.
 - **Multiply and Accumulate (MAC)**: realiza multiplicação e soma em **um único ciclo de clock**.
- Essas instruções **aceleram tarefas complexas de processamento de dados**, como as encontradas em **aplicações de áudio e vídeo**.
- Podem ter **opcionalmente** uma unidade de ponto flutuante (FPU).

Conjunto de instruções da família ARM

Cortex-M

Cortex-M4F

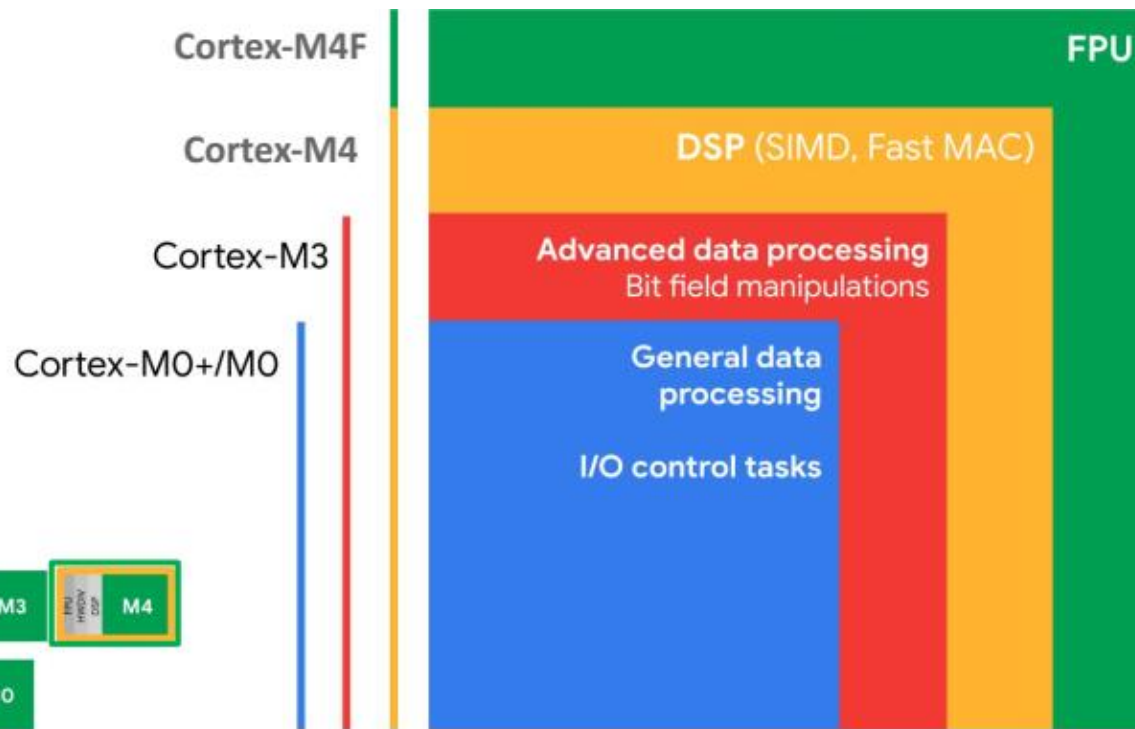
- Superconjunto contendo todas as instruções dos modelos anteriores.
- Diferença para o M4 é que *já vem com uma unidade de ponto flutuante (FPU)*.
- Como eles têm FPU, *possuem as instruções necessárias para cálculos em ponto flutuante*.



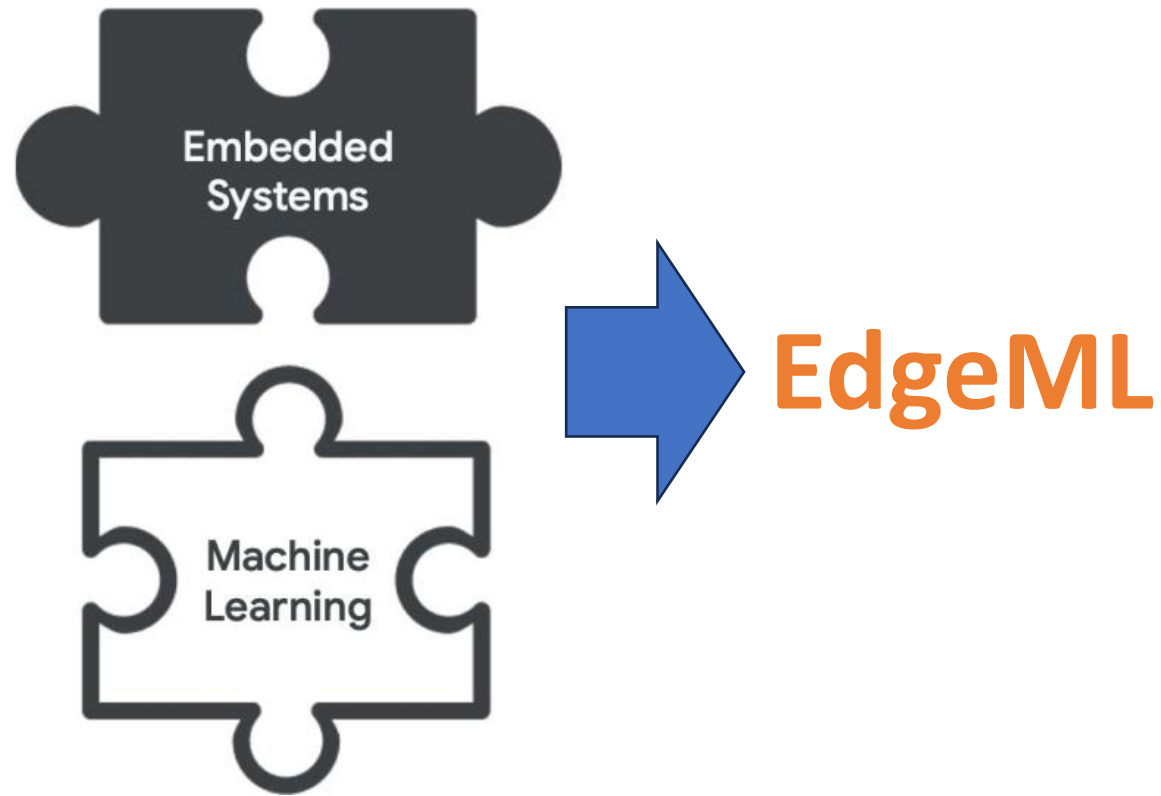
Conjunto de instruções da família ARM Cortex-M

Observações

- Até os modelos M3 e M4 não há FPU, todas as operações são feitas em **ponto fixo**.
- Com o EdgeML, evita-se ao máximo usar operações em **ponto flutuante**, pois **consomem mais memória e processamento, o que implica em mais consumo de energia**.
 - Uma variável de ponto flutuante de precisão simples ocupa 4 bytes.
 - O uso de operações em **ponto fixo** é uma das técnicas para reduzir o tamanhos de modelos de ML.

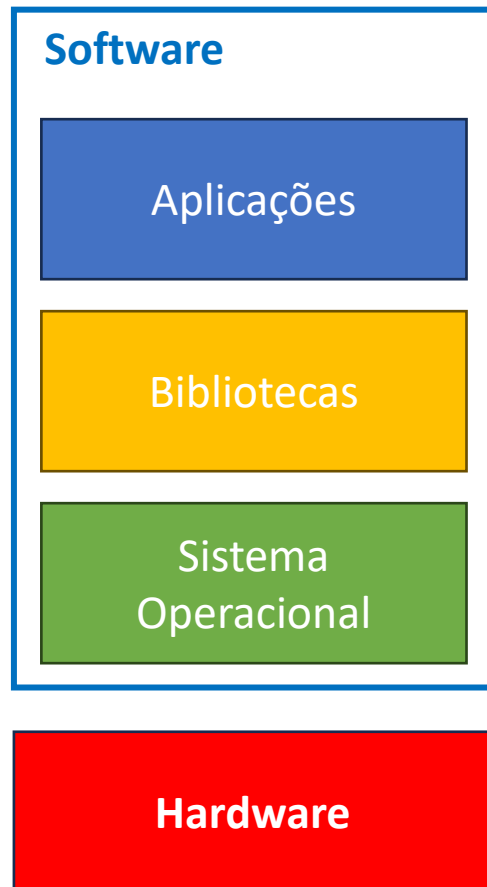


Desafios da execução de SW em sistemas embarcados



- Anteriormente, ***falamos dos desafios de hardware***, agora falaremos do ***software de um sistema computacional de uso geral para fazermos um paralelo com o mundo do EdgeML.***
- Na sequência, ***veremos os desafios para execução de SW***, incluindo algoritmos de ML, em dispositivos ***pequenos***, com ***restrições de custo, recursos computacionais e consumo.***

Software



- Vamos começar pelo sistema operacional (SO).
- Um sistema operacional **gerencia recursos e fornece uma interface entre os componentes físicos do computador (i.e., hardware) e os programas** que são executados nele.
- Ele atua como uma **camada de software intermediária** que permite que os usuários interajam com o computador e executem tarefas de forma eficiente.

Sistemas operacionais amplamente usados

Sistemas operacionais de uso geral



Sistemas operacionais móveis



Sistemas operacionais embarcados



Podemos dividir os OSs nos três grandes grupos ao lado.

Sistemas operacionais amplamente usados

Sistemas operacionais de uso geral



Sistemas operacionais móveis

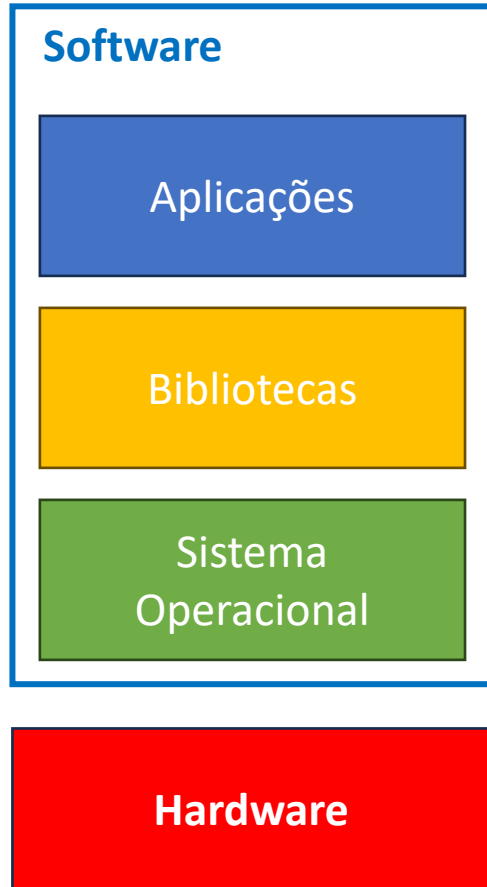


Sistemas operacionais embarcados



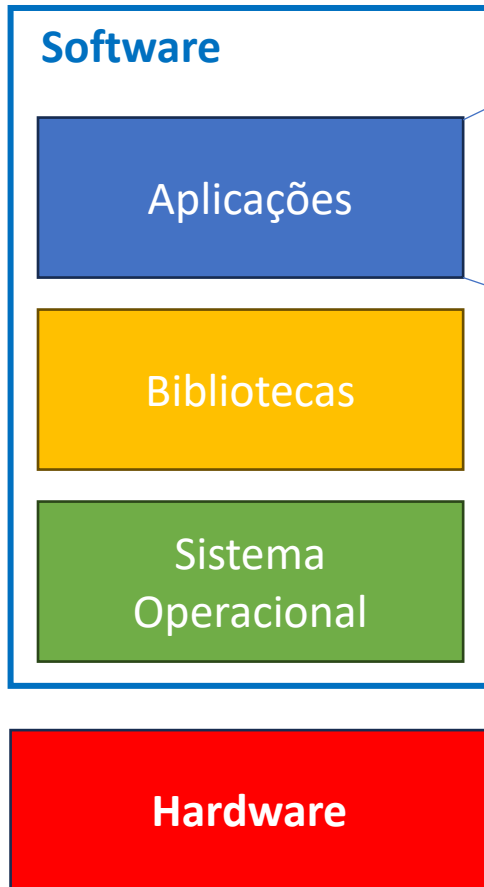
- Por exemplo, o arduino nano 33 pode rodar os SOs: **Free RTOS** e **ARM mbed**.
- Mesmo podendo usar um SO, em alguns casos, opta-se pela execução em “**bare metal**” (metal nu), i.e., sem SO, em cima do HW.
- O **bare metal** é usado quando se quer ter **controle direto sobre o HW**, criar **soluções extremamente específicas, eficientes e enxutas**.
 - Ex.: Aplicação *always on* para *keyword spotting*.

Bibliotecas



- Em cima do OS nós temos as bibliotecas.
- Bibliotecas são **conjuntos de código reutilizável** (funções, classes, etc.) para **adicionar funcionalidades a um programa sem a necessidade de escrever todo o código do zero**.
 - Por exemplo a biblioteca TensorFlow, que possibilita a criação e treinamento de modelos de ML.
 - Ou a biblioteca NumPy que fornece código para trabalharmos com matrizes multidimensionais (chamadas de *arrays*).

Bibliotecas



```
import numpy as np
```

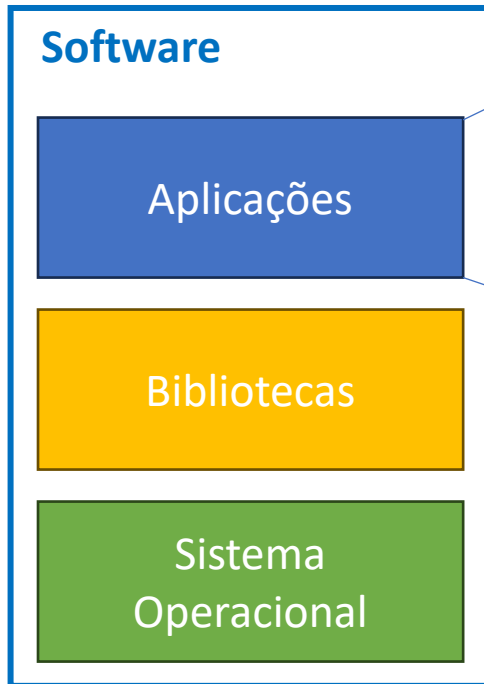
```
for x in range(10):  
    resultado = np.sqrt(x)  
    print(resultado)
```

```
0.0  
1.0  
1.4142135623730951  
1.7320508075688772  
2.0  
2.23606797749979  
2.449489742783178  
2.6457513110645907  
2.8284271247461903  
3.0
```

- A aplicação ao lado importa a biblioteca ***Numpy*** e utiliza a função ***sqrt()*** para calcular as raízes quadradas de 0 a 9.
- O resultado são valores em ***ponto flutuante (float)***.



Bibliotecas



```
import numpy as np
```

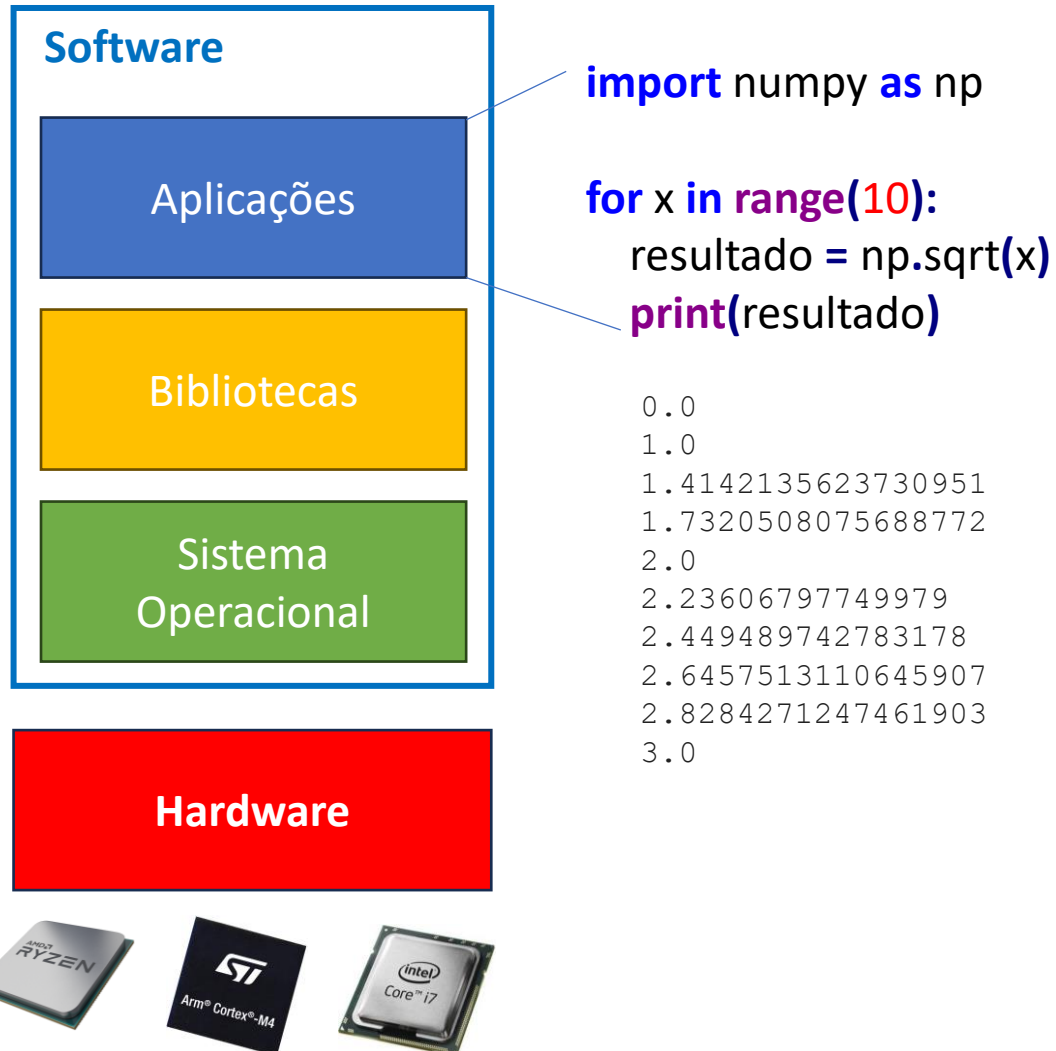
```
for x in range(10):  
    resultado = np.sqrt(x)  
    print(resultado)
```

```
0.0  
1.0  
1.4142135623730951  
1.7320508075688772  
2.0  
2.23606797749979  
2.449489742783178  
2.6457513110645907  
2.8284271247461903  
3.0
```

- Uma vez escrito o código, *devido à portabilidade do Python*, podemos *executá-lo virtualmente em qualquer sistema, com qualquer tipo de CPU*.
- Nós não precisamos nos preocupar com a portabilidade, *apenas em escrever e executar o código*.

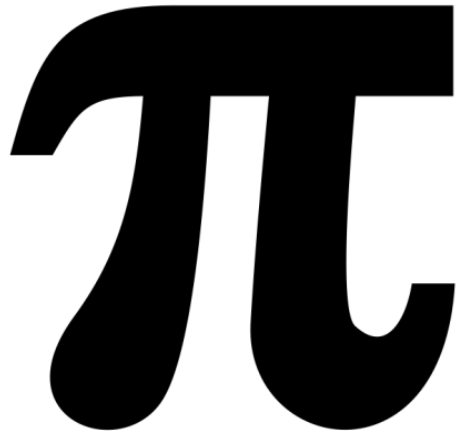


Bibliotecas



- A *máquina virtual* do Python se encarrega da *portabilidade*.
- Ela cria uma *camada de abstração* que *lida com os detalhes de baixo nível*.
- Desta forma, o código roda perfeitamente em qualquer SO e HW.
- Essa é a beleza de um sistema de uso geral.

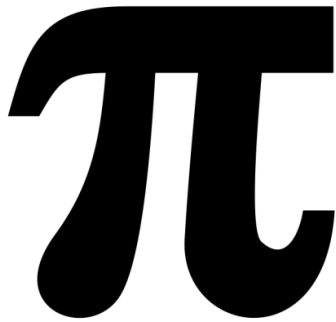
Portabilidade



```
3.141592653589793238462643383279502884197169399375105820974944592307816406286208998628034
82534211706798214808651328230664709384460955058223172535940812848111745028410270193852110
55596446229489549303819644288109756659334461284756482337867831652712019091456485669234603
48610454326648213393607280249141273724587006606315588174881520920962829254091715364367892
59036001133053054882046652138414695194151160943305727036575959195309218611738193261179310
5118548074462379627495673518857527248912279381830119491298336733624406506430860213849463
95224737190702179860943702770539217176293176752384674818467669405132000568127145263560827
78577134275778960917363717872146844090122495343014654968537105079227968925892354201995611
21290219608640344181598136297747713099605187072113499999983729780499510597317328160963185
95024459455346908302642522308253344685035261931188171010003137838752886587533208381420617
17766914730359825349042875546873115956286388235378759375195778185778053217122680661300192
78766111959092164201989380952572010654858632788659361533818279682303019520353018529689957
73622599413891249721775283479131515574857242454150695950829533116861727855889075098381754
63746493931925506040092770167113900984882401285836160356370766010471018194295559619894676
78374494482553797747268471040475346462080466842590694912933136770289891521047521620569660
24058038150193511253382430035587640247496473263914199272604269922796782354781636009341721
64121992458631503028618297455570674983850549458858692699569092721079750930295532116534498
7202755960236480665499119881834797753566369807426542527862551818417574672890977727938000
81647060016145249192173217214772350141441973568548161361157352552133475741849468438523323
90739414333454776241686251898356948556209921922218427255025425688767179049460165346680498
8627232791786085784383827967976681454100953883786360950680064225125205117392948960841284
88626945604241965285022210661186306744278622039194945047123713786960956364371917287467764
65757396241389086583264599581339047802759009946576407895126946839835259570982582262052248
94077267194782684826014769909026401363944374553050682034962524517493996514314298091906592
50937221696461515709858387410597885959772975498930161753928468138268683868942714155991855
92524595395943104997252468084598727364469584865383673622262609912460805124388439045124413
6549762780797715691435997700129616089441694888558484063534220722258284886481584560285060
16942739452267467678895252138522549954686727823986456596116354886230577456498035593634568
174324112515078694794510965960940252287971089314566913686722874894056010150330861792868
09208747609178249385890097149096759852613655497818931297848216829898487226588048575640142
7047755513237964145152374623436454285844479526867821051141354735739523113427166102135969
5362314429524849371871101457654035902799344037420073105785390621983874478084784898332144
57138687519435064302184531910484810053706146806749192781911979399520614196634287544406437
45123718192179989391015919561814675142691239748940907186494231961567945208095146550225231
6038819301420937621378559566389377808303906979207734672218256259966150142150306803844773
45492026054146659252014974428507325186660021324340881907104863317346496514539057962685610
05508106658796989163574736384052571459102897064140110971206280439039759515677157700420337
86993600723055876317635942187312514712053292819182618612586732157919841464882916447060957
52706957220917567116722910981690915280173506712748583222871835209353965725121083579151369
8820914442100675103346711031412671113699086585163983150197016515116857143765761835155650
8849098985998238734552833163550764791853589322618548963213293308985706420467525907091548
1416549859461637180270981994309924488957571282890592323326097299712084433573265483823911
9325974636673058360414281388303203824903758985243744170291327656180937344403070746921120
19130203303801976211011004492932151608424448596376698389522868478312355265821314495768572
6243344189303968642624341077322697802807318915441101044682325271620105265227211660396665
57309254711055785376346682065310989652691862056476931257058635662018558100729360659876486
1179104533488503461136576867532494416680396265797871855608455296541266540853061434443185
8676975145661406800700237877659134401712749470420562230538994561314071127004078547332699
```

- *Portabilidade* é *algo que damos como certo e que não pensamos muito a respeito* quando criamos códigos para sistemas de uso geral.
- Porém quando partimos para *sistemas embarcados, isso se torna um problema.*
- Vamos ver um problema envolvendo cálculos em ponto flutuante: o cálculo de π .

Portabilidade

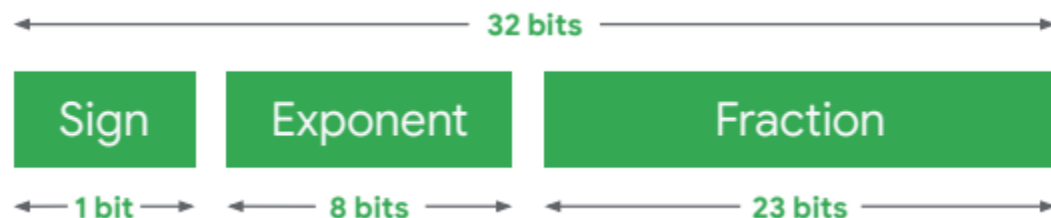


```
3.14159265358979323846264338327950288419716939937510582097494459230781640620620989682034
8253421170679821480865132823066470938446095058223172535940812848111745028410270193852110
55596446229489549303819644288109756659334461284756482337867831652712019091456485669234603
48610454326648213393607260249141273724587070060631558817489152092092829254091715364367862
59036001133053054882046621384146051841516094330572703657596919030921861173810261170310
51185480744623799627495673518857527248912279381830119491298336733624406566430860213949463
95224737190702179860943702770539217176293176752364674818467669405132000568127145263506827
78577134275778960917363717872146844090122465430146549585371050192278689588234201995611
212902196086403441815981362977477130960518707211349999998329780499510597317328160963185
950244594553469083026425220825334468803526193118817101000313783875288658753208381420817
1776691473035982534984267554687315565266386233378759375195781657805321712280661300192
78766111569092164201989380952720106548563788626315338182786230301825035018526696957
7362259941389124972177528347913155748572424541506959508295331168617278558907508381754
6374649393192506940092770167113900894862401258361603563707601047101819429559619894676
783744944625379747268471040475346420040668429069491293313677028891921047521620509690
240580381501935112533824300358764024749647326391419927260426992279678234781636009341721
641219924586315030286182974555706749838505494585869269956909272107975093029532116534498
7202759802364906549911981834797753566369807420542527862551818417574072890977772930000
816470600161452491921732172147723501414419735654816136115735255213347574184946843852323
9073941433345477624168625189835694856209921922218427255025425688767179049460165346680498
862723278178608578430302796797668145410095383786309506900422512505117392949890841284
886265456042419652850222106611863067442786220391945404712371378696086363437191297467764
6575739624138908658326459958133904780275900994657640789512694683983525957098258226205248
9407726719478268452601476990902640136394437453050682034962524517439396514314298091906592
593372216964615157086387410978689597729549803016173302468138286638639427415991855
925495395943104997252468084598727364469584653637622262609912460805124388439045124413
654976278077715691435997007001296160894416946855584840635342072225824886481584560285060
1844273945226746788952513852349854667278239649596116354862305745649803553634568
17432411251507606947945109659604052288797108931456691368672287489405601015033861792868
0920874760917824938589009714909675985261365549781893129794821682998948722658048575640142
704775551323796414515237462343645285844746268967821051141354735739523113427166102135969
53623144295245493718711047654059607583403742007310786389621883874478984784896332144
571386875194350643021845319104848100537061468067491927819119793952061419663428754406437
4512371819217999391015919561814675142691239748940907186494231961567945208095146550225231
60388193014209376213785596638937787830360679207734672182662996615014215030603844773
4549202605414665925201497442850732518666002132434088190710486331734649651453905796268610
0550810665879699816357473638405257145910289706410010971206280439039759515677157700420337
869030072305576317635942187125147120532928191626186125673215791894148482916447069697
5270695722091756711672291098169091528017350671274858322871835209353665725121083579151369
88209144421006751033467110314126711369908658516396315019701651511688517143765761835155650
884909898698923873455283316350764791853589322618548963213293308985708420467525907091548
141654895945163710027089189430952448895712528959252333697259712084355732654683823811
9325974636673058360414281388303203824903758985243441702913276561809377344403070746922120
19130203038019762110100449252151608424448596376983895226847831235265821314495768572
624334118930398964262434107322697802807318915441101044623327162010526522711168036665
573092547110557853763468206531098965269186205647693125705863562018558100729360659876486
11791045334885034611365768675324944166803962657978771855608452965412665408306143443185
86789751458614068007002378776991344017127494704205622305389945613140711270004078547332699
```

- Suponhamos um código com o seguinte cálculo:

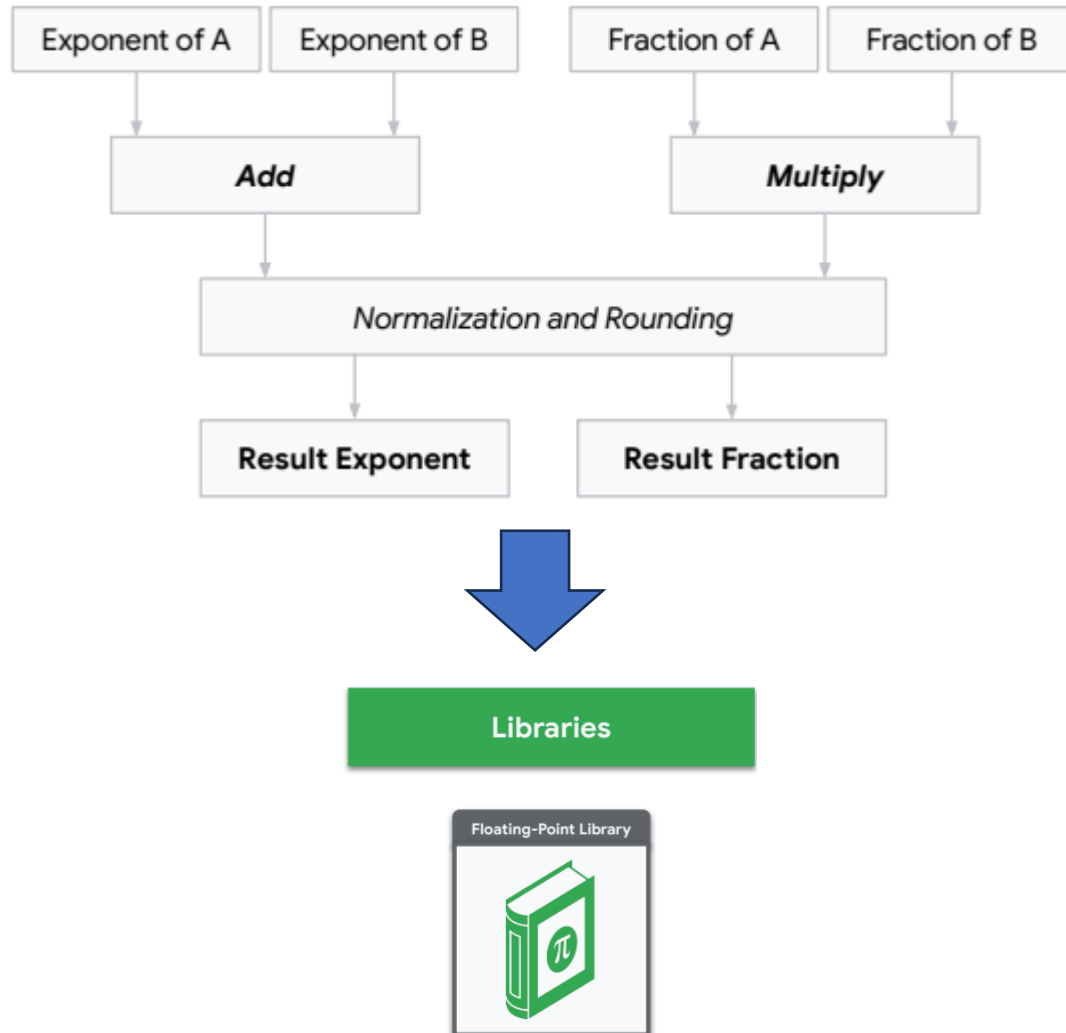
- $22/7 = 3.14159265359...$

- A resposta é um **valor em ponto flutuante**, que é tipicamente expresso como 3 componentes: **sinal**, **expoente** e **mantissa** (ou fração).



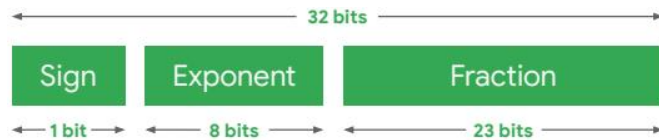
- Operações com valores em ponto flutuante **são complexas e, consequentemente, computacionalmente custosas.**

Portabilidade

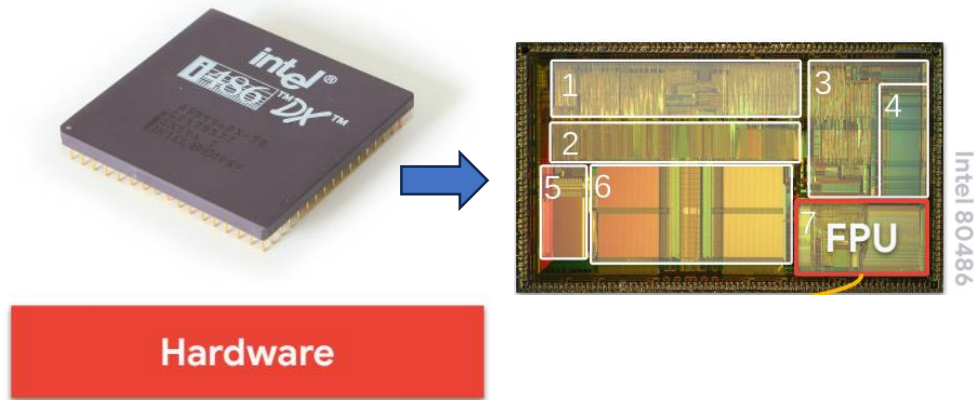


- Como forma de **abstrair a complexidade dos cálculos e facilitar o uso** de operações em ponto flutuante, podemos **implementá-las como uma biblioteca** que teria funções para adição, multiplicação, etc. destes valores.
- Entretanto, uma **biblioteca puramente feita em SW** para tal finalidade tende a ser **muito lenta**.

Portabilidade



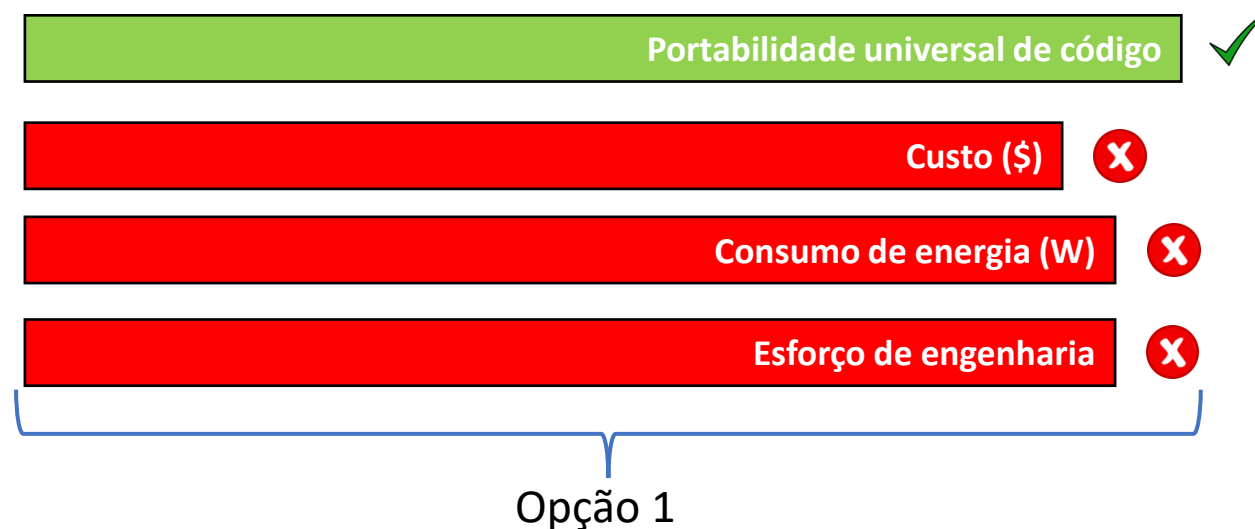
Single Precision
IEEE 754 Floating-Point Standard



Até as CPUs 386, o coprocessador matemático, contendo a FPU, era um *chip* separado instalado opcionalmente em um *socket* da placa mãe.

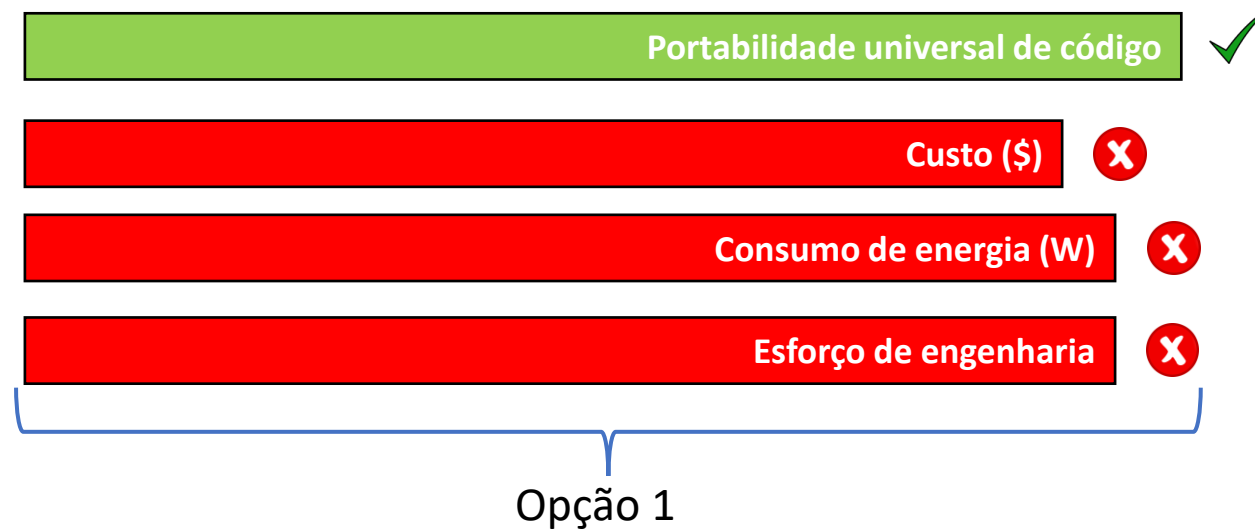
- Ao longo dos anos, para acelerar essas operações, o que ocorreu foi a criação de *partes da CPU especializadas em operações de ponto flutuante*, as chamadas *Floating Point Units (FPUs)*.
- Isso foi ótimo porque desde sua *proliferação, praticamente toda CPU tem uma FPU dentro dela*.
- Portanto, *escrevemos o código uma única vez e ele é executado em qualquer lugar*.
- Mas isso expõe uma *relação de compromisso*.

O perde-e-ganha da portabilidade



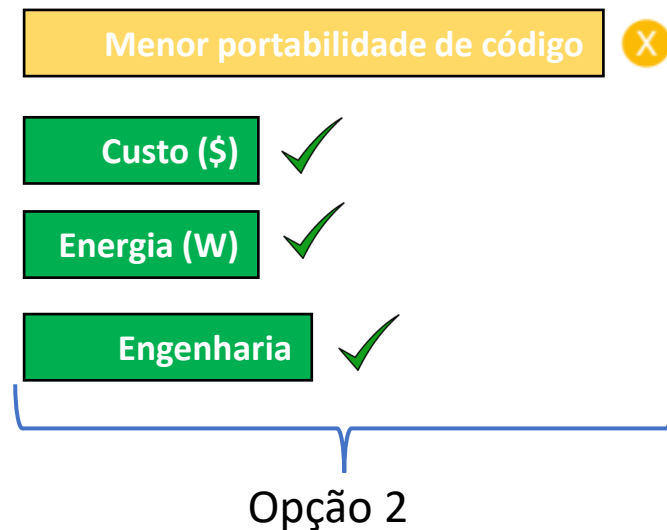
- A **portabilidade universal de código têm altos custos** associados.
- Sua **implementação custa caro** (HW complexo e mais caro).
- **Mais energia é consumida** por esses sistemas de uso geral (W).
 - Por exemplo, a FPU quando usada aumenta o consumo de energia da CPU.

O perde-e-ganha da portabilidade



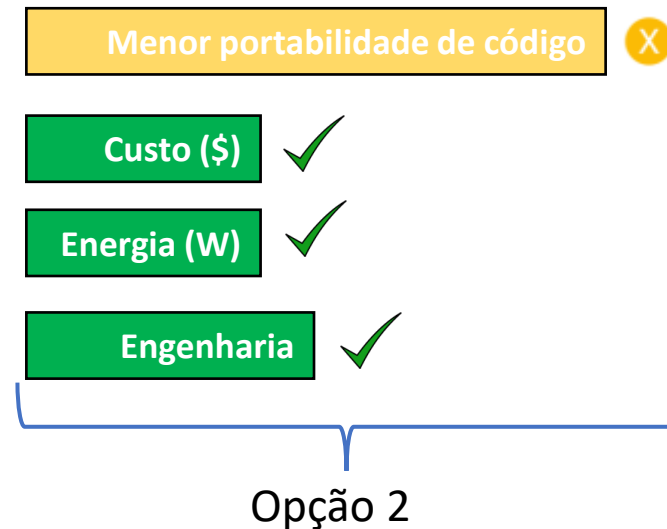
- Necessita-se de ***um esforço de engenharia (i.e., de projeto) grande*** para construí-los e que, conseqüentemente, ***eleva*** mais ainda ***os custos***.
- Portanto, se queremos portabilidade universal, temos que pagar estes preços.

O perde-e-ganha da portabilidade



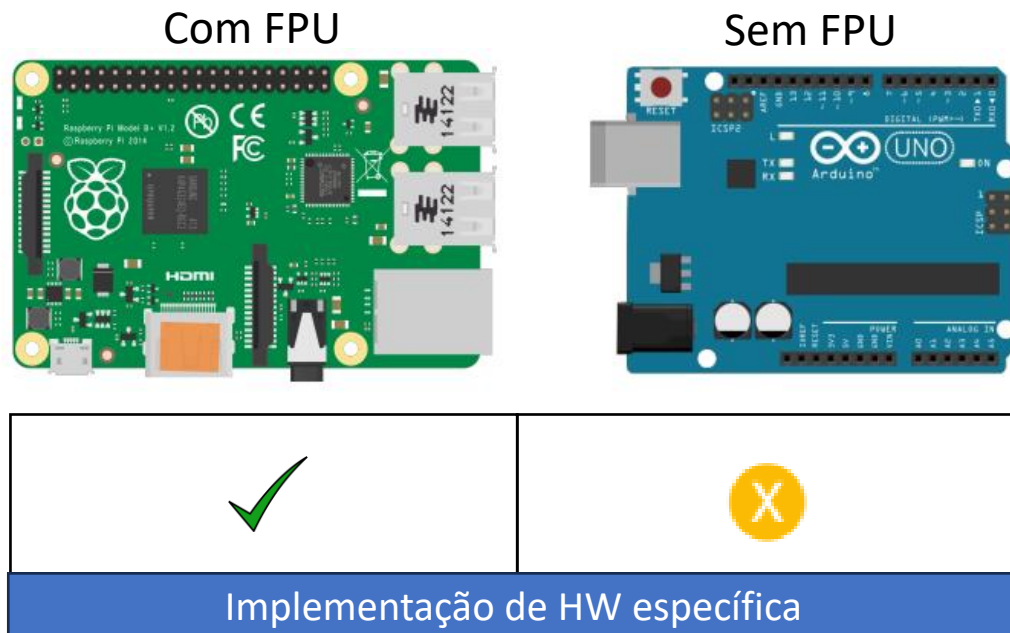
- Agora digamos que queremos trabalhar com *sistemas embarcados*.
- Eles são por natureza
 - *baratos*,
 - *pequenos*,
 - *eficientes energeticamente*,
 - *enxutos* em termos de memória e poder computacional e
 - *fáceis de serem projetados*.
- Essas características são ótimas para um sistema embarcado.

O perde-e-ganha da portabilidade



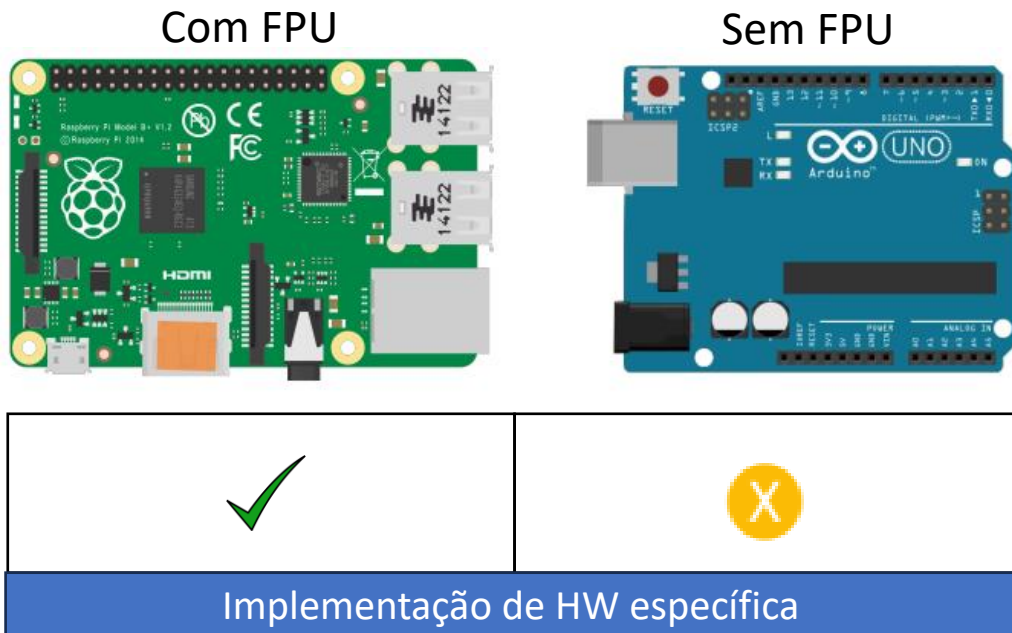
Porém, uma implicação direta delas é que a ***preocupação com a portabilidade universal é mínima ou inexistente.***

O perde-e-ganha da portabilidade



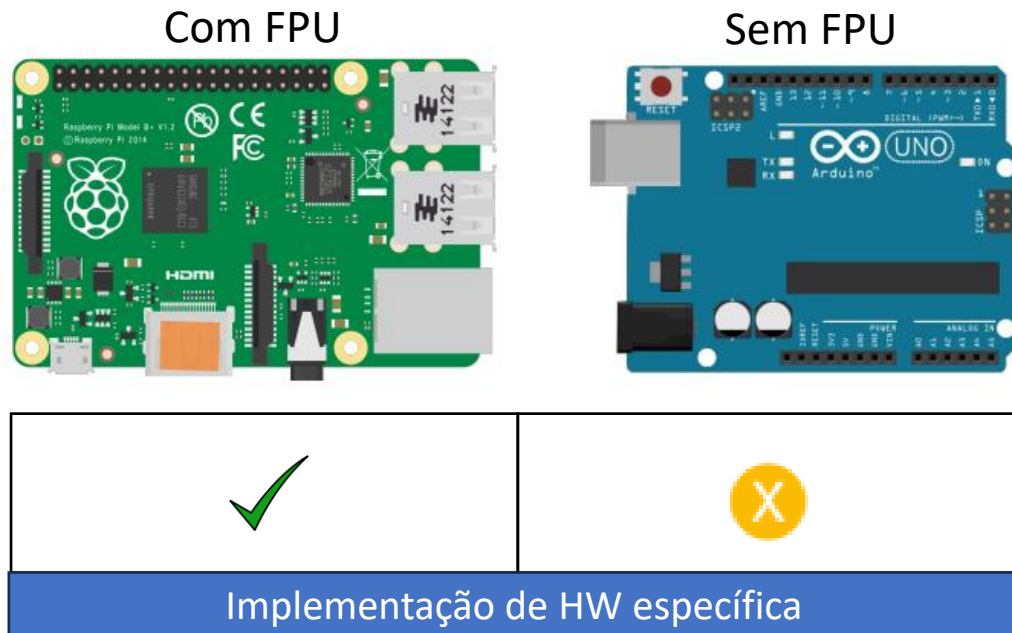
- Por exemplo, o que acontece se quisermos rodar um código em outro sistema e nele não tivermos os recursos que o primeiro sistema tinha e que utilizamos no nosso código, como, por exemplo, uma FPU?
- Outros exemplos de recursos que podem ser exclusivos são:
 - Sensores, atuadores, conversores AD/DA, etc.
- *O código não irá ser executado!*

O sacrifício da portabilidade



- O *código especializado para um sistema embarcado* ***não é portátil*** para outro.
- Devido à limitação de recursos, ***sacrifica-se a portabilidade*** entre sistemas ***para obter-se***:
 - **Otimização do desempenho**: o ***código é adaptado à arquitetura de HW*** específica do sistema, resultando em ***código altamente eficiente e rápido***, ***porém menos portátil***.
 - **Otimização de memória**: otimizações para ***economia de memória*** podem exigir ***estruturas de dados específicas e técnicas de programação menos portáteis***.

O sacrifício da portabilidade



- **Eficiência Energética:** *otimizar* o código para *minimizar o consumo de energia* pode envolver *estratégias que não são necessariamente portáveis*.
- Portanto, esse sacrifício levanta uma questão fundamental:
 - *Como habilitamos o EdgeML em sistemas embarcados tão diferentes se a portabilidade é baixa ou inexistente?*
- Esse é um desafio crítico e que veremos como abordá-lo ao longo do curso.

Resumo do que vimos até agora

- *Hardware embarcado* é ***extremamente limitado*** em termos de ***poder computacional, consumo de energia e armazenamento.***
- *Software embarcado* ***não é tão portátil e flexível*** quanto o software para sistemas de computação de uso geral.

Essas ***limitações*** são ***comuns*** em ***sistemas embarcados*** devido ao ***objetivo*** de ***otimização para tarefas específicas*** e ***eficiência no uso dos recursos.***

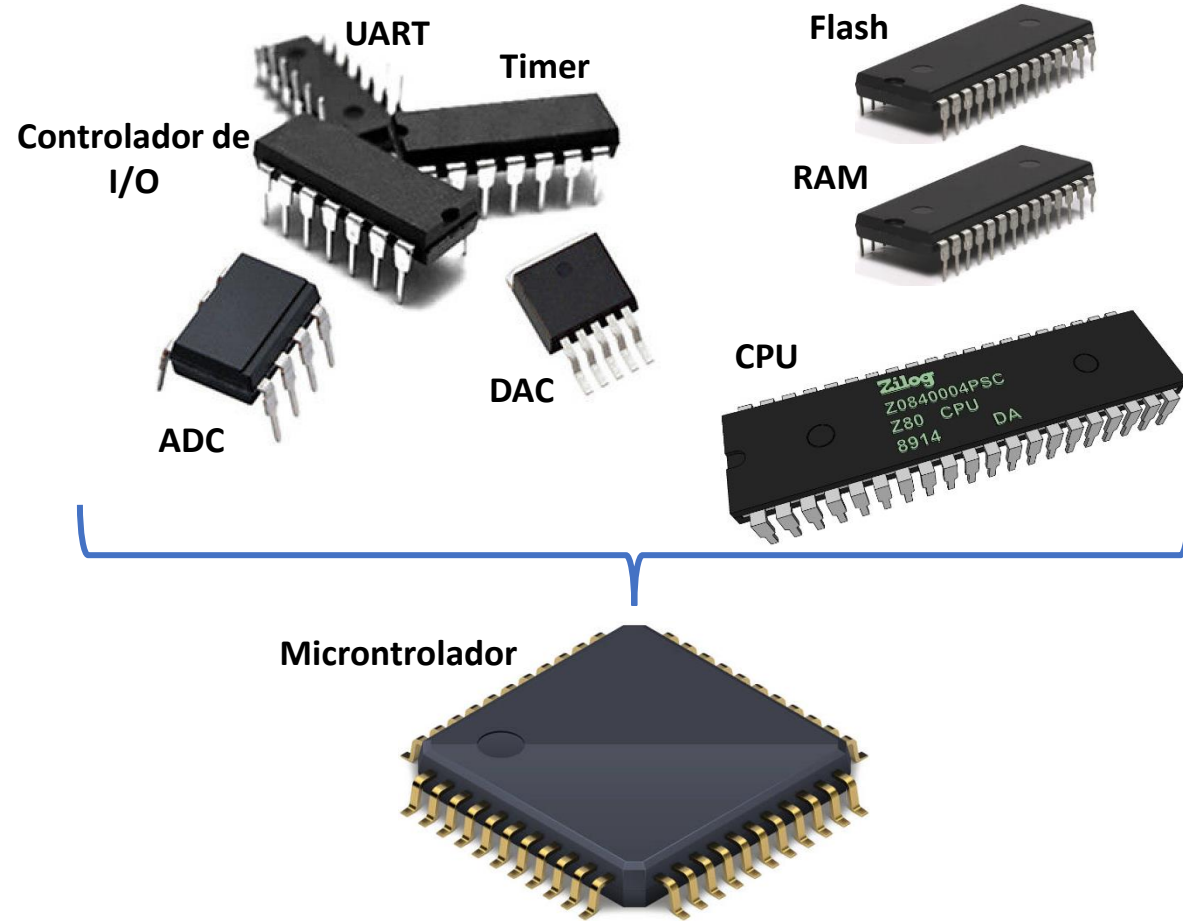
Isso ocorre porque o ***SW embarcado*** é ***altamente adaptado ao HW*** e aos ***requisitos específicos da aplicação***, resultando em uma ***menor capacidade de reutilização*** (portabilidade e adaptação) a diferentes plataformas e contextos.

Atividades

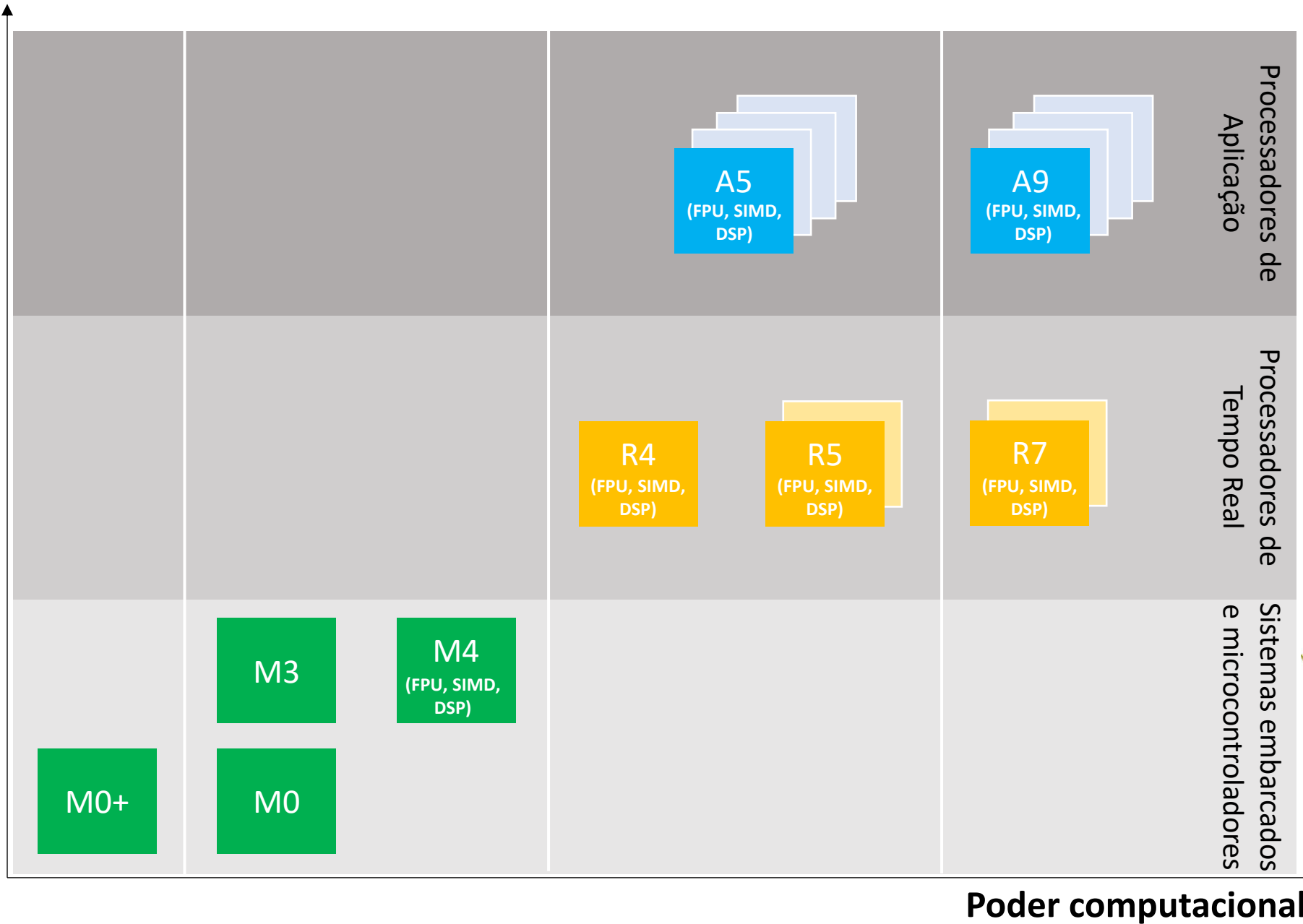
- Quiz: “***TP557 – Desafios do TinyML – Sistemas Embarcados***”

Perguntas?

Obrigado!



Consumo



Poder computacional

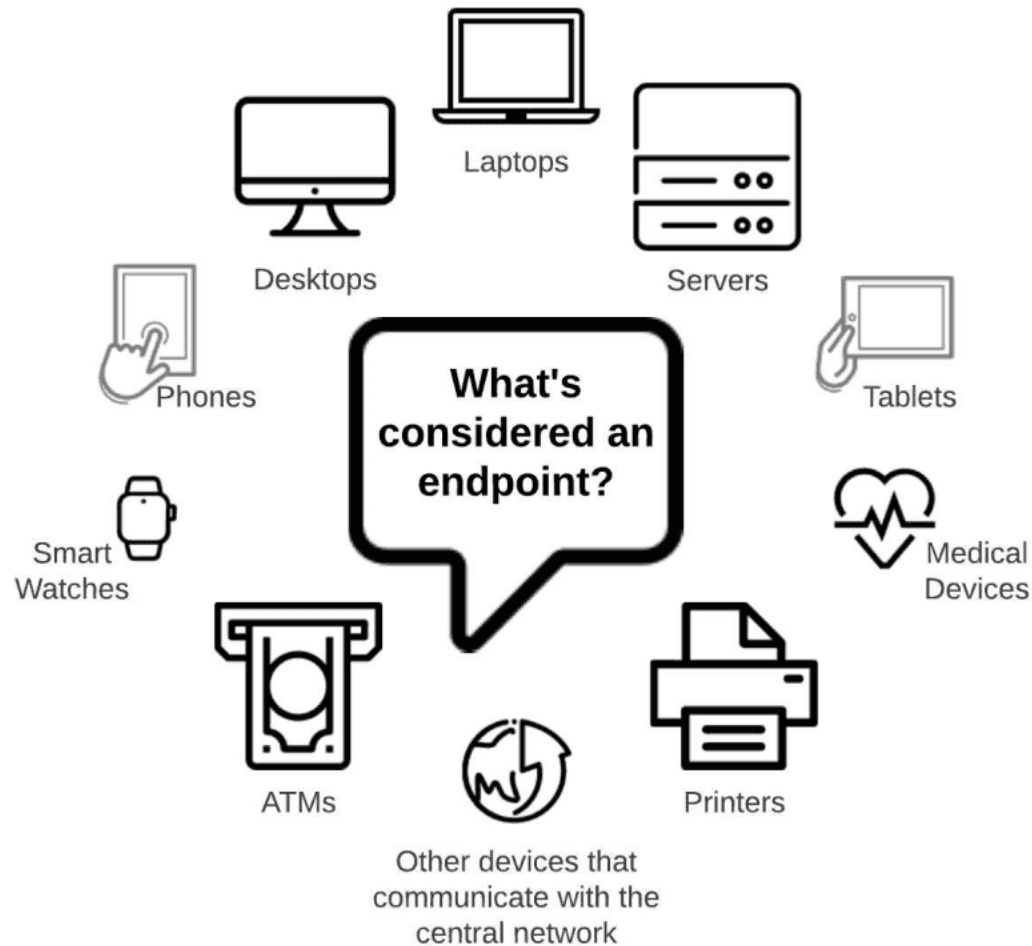
Processadores de Aplicação

Processadores de Tempo Real

Sistemas embarcados e microcontroladores



Endpoint



- Um *endpoint* é um dispositivo de computação remota que se comunica com uma rede ou com a Internet.
- Em outras palavras, se um dispositivo estiver conectado a uma rede, ele será considerado um *endpoint*.